KOSTA AI Lab · AI Native SDLC

Spec Kit으로 실전 소프트웨어 설계

멀티셀러 마켓플레이스를 설계 산출물로 그려냅니다

설계가 코드보다 먼저 · TypeScript와 Hono

SECTION

00 소프트웨어 설계 개론

- 설계가 요구사항과 구현 사이의 결정임을 이해합니다
- 좋은 설계의 기준과 AI 시대 설계의 위상을 짚습니다
- 오늘 만들 산출물이 설계의 어느 층위에 놓이는지 살펴봅니다

오전 도입 · 설계란 무엇이며 왜 코드보다 먼저인가

소프트웨어 설계란

요구사항을 코드로 옮기기 전에 구조·관계·흐름·경계를 정하는 결정의 묶음입니다



💡 설계는 코드가 아니라 판단입니다. 같은 요구사항도 설계에 따라 유지보수성·확장성·정합성이 갈립니다.

좋은 설계의 기준

'지금 빨리'가 아니라 '나중에 바꾸기 쉽게'를 겨냥하는 다섯 가지 잣대입니다

구조 **높은 응집도**	한 단위가 하나의 책임에 집중합니다. 관련된 일이 흩어지지 않고 한곳에 모입니다.
의존 **낮은 결합도**	단위 사이의 의존을 줄여 변경이 번지는 범위를 좁힙니다.
변화 **변경 용이성**	요구가 바뀔 때 전체를 흔들지 않고 국소 수정으로 대응합니다.
책임 **명시적 경계**	무엇이 시스템 안이고 무엇이 밖인지 분명합니다. 안은 구현하고 밖은 연동합니다.
선택 **트레이드오프 인식**	정답이 아니라 맥락에 맞는 선택임을 압니다. **예시:** 재고를 미리 선점할지, 결제 뒤에 차감할지

AI 시대 설계의 위상

AI가 구현을 빠르게 대신할수록 사람이 맡을 몫은 설계 결정으로 좁혀집니다

01 병목이 이동합니다

AI가 코드를 빠르게 뽑을수록 느린 지점은 타이핑이 아니라 무엇을 어떻게 만들지 정하는 설계 결정입니다.

02 지시하려면 쥐어야 합니다

AI에게 '무엇을·어떤 구조로·왜'를 지시하려면 사람이 설계를 이해하고 손에 쥐고 있어야 합니다.

03 결정을 남깁니다

오늘은 이 설계 결정을 Spec Kit 단계마다 다이어그램으로 남기고, 그 의미를 해석합니다.

🧭 코딩은 설계의 결과일 뿐입니다. 설계와 판단은 사람이 맡습니다.

오늘의 프로젝트 · 개념

오늘 만들 멀티셀러 마켓플레이스

여러 판매자가 한 플랫폼에 입점하는 e-커머스 마켓입니다. 2편에서 세운 상품·주문·재고 모델을 이 과정에서 판매자·결제·배송·정산·환불까지 넓힙니다

한 주문, 여러 판매자

구매자는 판매자 A와 B의 상품을 한 번에 담아 한 번 결제합니다. 시스템은 이를 판매자별로 나눠 배송·정산·환불까지 각각 책임집니다. 오늘의 모든 설계 산출물은 이 하나의 마켓을 그립니다.

1. **구매자**
   여러 판매자 상품을 한 번에 주문하고 결제합니다
2. **판매자**
   입점해 상품을 등록하고 주문을 각자 배송합니다
3. **관리자**
   입점 승인과 정책·분쟁을 관리합니다
4. **배송사**
   판매자별로 나뉜 주문을 단위로 배송합니다
5. **결제 게이트웨이**
   결제 승인과 취소를 처리하는 외부 시스템입니다
6. **정산 시스템**
   수수료를 공제해 판매자에게 지급합니다
7. **CS**
   취소·환불·분쟁을 중재합니다

오늘의 프로젝트 · 설명

설계가 필요한 여섯 요구사항

쉬워 보이는 기능마다 까다로운 규칙이 숨어 있습니다. 요구사항 하나하나가 오늘 그릴 산출물과 설계 결정(ADR), 추적성 매트릭스의 R1~R6으로 이어집니다

멀티셀러 주문 분할	한 주문을 판매자별 SubOrder로 나눠 배송과 정산을 각자 책임지게 합니다.	ADR-001
결제 전 재고 선점	결제가 확정되기 전에 재고를 먼저 선점해 주문이 몰려도 초과 판매를 막습니다.	ADR-002
멱등 결제	재시도가 겹쳐도 결제 승인은 한 번만 일어나도록 멱등키로 보장합니다.	ADR-003
판매자별 배송	분할된 주문 단위로 배송하고 배송 완료 시점을 상태로 남깁니다.	ADR-004
수수료 정산	배송 완료분만 모아 수수료를 공제하고 판매자에게 지급합니다.	ADR-004
환불 역산	취소가 생기면 재고를 복원하고 이미 잡은 정산을 되돌립니다.	ADR-004

설계의 네 층위

위에서 아래로 구체화됩니다. 요구사항이 구조를 규정하고 구조가 행위를 낳고 행위가 아키텍처로 배치됩니다

층위	던지는 질문	오늘의 산출물
요구사항	누가 무엇을 원하나	유스케이스 다이어그램
구조	무엇으로 이뤄지나	도메인 모델 · ERD
행위	어떻게 흐르나	시퀀스 · 상태머신
아키텍처	어디서 도나	컴포넌트(C4)

오늘의 설계 산출물 지도

여섯 다이어그램이 네 층위를 위에서 아래로 채웁니다. 요구사항이 구조를, 구조가 행위를, 행위가 아키텍처를 규정합니다

요구사항
누가 무엇을 원하나
- 유스케이스 다이어그램

구조
무엇으로 이뤄지나
- 도메인 모델
- ERD

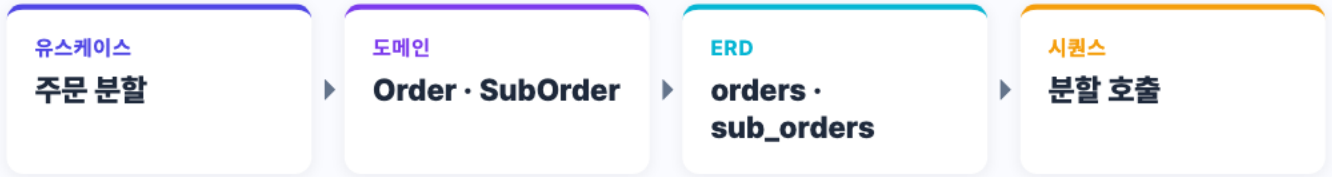
행위
어떻게 흐르나
- 시퀀스 다이어그램
- 상태머신

아키텍처
어디서 도나
- 컴포넌트(C4)

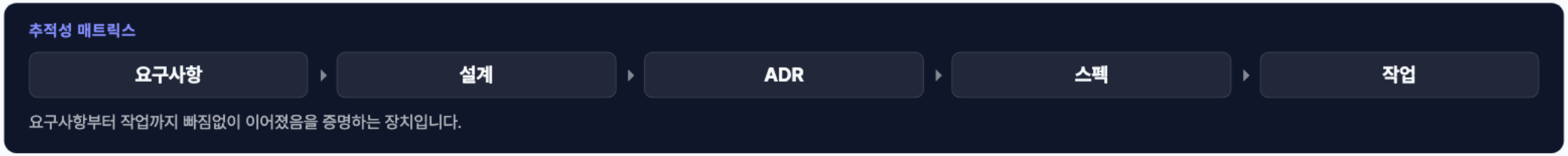
🔗 여섯 다이어그램이 네 층위를 채우고 ADR과 추적성 매트릭스가 이 층위들을 가로질러 잇습니다.

산출물을 잇는 추적성

각 다이어그램은 독립적이지 않고 앞 산출물의 결과입니다. 한 결정이 산출물을 관통해 이어집니다



하나가 바뀌면 영향이 어디까지 가는지, 추적성으로 확인합니다.



오늘의 여정

여덟 블록이 Spec Kit 단계를 따라 설계 산출물을 하나씩 채웁니다. 지금은 그 문을 여는 개론입니다

지금 설계 개론 설계란 무엇이며 왜 코드보다 먼저인가를 세우고 여덟 블록으로 들어갑니다

1	09:00	/constitution	왜 지금 설계인가 · 저장소 세팅	시스템 경계
2	10:00	/specify ①	요구사항에서 멀티 액터 유스케이스로	유스케이스
3	11:00	/specify ②	도메인 개념 모델링	도메인 모델
4	13:00	/plan ①	데이터 설계	ERD
5	14:00	/plan ②	상호작용과 상태	시퀀스 · 상태머신
6	15:00	/plan ③	구조와 아키텍처 결정	컴포넌트(C4) · ADR
7	16:00	/tasks	요구사항·설계·작업 추적	추적성 매트릭스
8	17:00	/implement	이슈에서 PR·머지까지 · 회고	Git 흐름

설계 개론 · 체크포인트

도입 정리 (설계란 무엇이며 무엇을 남기는가)

- ✔ 소프트웨어 설계는 요구사항과 구현 사이에서 내리는 결정이며 코드가 아니라 판단입니다.
- ✔ 응집도·결합도·변경 용이성·명시적 경계·트레이드오프로 설계의 품질을 판단합니다.
- ✔ AI가 구현을 대신할수록 병목은 설계로 옮겨가고 설계와 판단이 사람의 자리가 됩니다.
- ✔ 설계는 요구사항·구조·행위·아키텍처의 네 층위로 나뉘고 오늘의 여섯 다이어그램이 각 층위를 채웁니다.
- ✔ 산출물은 서로 이어지며 추적성 매트릭스가 요구사항부터 작업까지 빠짐없이 이어졌음을 증명합니다.

SECTION

01 Constitution과 Spec Kit 워크플로우

- 설계를 코드에 숨기지 않고 산출물로 남겨야 하는 이유를 짚습니다
- Spec Kit 다섯 단계와 constitution 원칙을 짚습니다
- 시스템 경계 개관과 GitHub 개발 흐름을 함께 봅니다

50분 (블록 1) · /constitution

CONSTITUTION · 설계를 남기는 방식

설계를 코드가 아니라 산출물로

개론에서 본 '설계 병목'을 실무에서는 코드에 숨기지 않고 산출물로 남겨 풉니다

BEFORE · 코드 먼저

구현이 설계를 대신하던 방식

- ✕ 요구사항을 머릿속에만 두고 바로 코딩합니다
- ✕ 구조·데이터·상태 결정이 코드 곳곳에 숨어 흩어집니다
- ✕ AI가 빠르게 짜줄수록 검증할 설계 근거가 없습니다
- ✕ 왜 이렇게 짰는지 나중에 아무도 설명하지 못합니다

전환

AFTER · 설계 먼저

산출물로 결정을 남기는 방식

- ✓ 유스케이스·도메인·ERD로 결정을 먼저 확정합니다
- ✓ 설계 결정을 다이어그램에 남깁니다
- ✓ 명확한 설계를 근거로 AI에게 구현을 지시합니다
- ✓ ADR과 추적성으로 왜 이 형태인지 설명할 수 있습니다

CONSTITUTION · 오늘의 관통 사례

멀티셀러 마켓플레이스 한 바퀴

주문 분할부터 정산·환불 역산까지, 하나의 도메인으로 모든 설계를 잇습니다

1. **여러 판매자 상품을 한 주문에 담아 결제**
   구매자는 판매자 A와 B의 상품을 한 번에 주문하고 한 번 결제합니다.
   구매자 · 결제

2. **판매자별로 주문을 분할하고 재고를 선점**
   주문을 판매자별 SubOrder로 나누고 결제 전에 재고를 선점해 오버셀을 막습니다.
   SubOrder · reserve

3. **판매자가 각자 배송하고 배송 완료를 기록**
   SubOrder 단위로 배송을 접수하고 markDelivered로 완료를 표시합니다.
   배송 · markDelivered

4. **배송 완료분만 정산 대상으로 집계**
   정산 주기에 완료된 SubOrder를 집계해 수수료를 공제하고 판매자에게 지급합니다.
   정산 · 수수료

5. **부분 취소·환불 시 재고와 정산을 역산**
   이미 정산된 건은 차기 정산에서 차감하고 취소된 재고는 복원합니다.
   환불 · 역산

CONSTITUTION · SPEC KIT 5단계

스펙에서 구현으로 이어지는 다섯 커맨드

각 커맨드는 앞 단계의 산출물을 근거로 실행하며 clarify·checklist·analyze 보조 단계로 품질을 점검합니다

1 **/speckit.constitution**
프로젝트의 원칙과 제약을 먼저 정합니다. 실행하면 constitution.md가 생성되고 이후 모든 단계가 이 원칙을 따릅니다.

2 **/speckit.specify** clarify
무엇을 만들지 자연어로 설명하면 요구사항 스펙이 spec.md에 나옵니다. 이어 clarify로 모호한 부분을 되물어 스펙을 다듬습니다.

3 **/speckit.plan** checklist
스펙을 어떻게 구현할지 설계합니다. 데이터 모델·상호작용·구조를 plan.md에 담고 checklist로 설계 누락을 점검합니다.

4 **/speckit.tasks** analyze
계획을 실행 가능한 작업 목록으로 분해해 tasks.md에 담습니다. analyze로 요구사항·설계·작업의 정합성을 검사합니다.

5 **/speckit.implement**
작업을 하나씩 구현합니다. 이슈에서 시작해 코드와 테스트를 만들고 PR로 main에 병합합니다.

Spec Kit, github.com/github/spec-kit (specify 0.12.8)

CONSTITUTION · 개념

설계를 좌우하는 아홉 원칙

2편에서 세운 다섯 원칙 위에 이 과정의 멀티셀러 원칙 넷을 더합니다. 이 아홉 원칙은 유스케이스·데이터·상태·구조 설계의 판단 기준입니다.

2편 계승 · I~V 단일 판매자 API에서 이어받은 토대

I **스펙 우선**
코드보다 스펙을 먼저 씁니다. 무엇을·왜 하는지 동작으로 정의합니다.

II **테스트 우선**
요구사항마다 성공과 실패 경로를 테스트로 짝짓습니다.

III **얇은 저장소**
저장소를 인터페이스 뒤에 두어 언제든 교체할 수 있게 만듭니다.

IV **명시적 에러**
예외는 알맞은 상태코드와 JSON 에러 본문으로 분명히 알립니다.

V **서버 권위**
합계·재고·수수료 같은 파생값은 서버가 계산합니다.

이번 편 추가 · VI~IX 멀티셀러로 넓히며 새로 못 박은 원칙

VI **SubOrder 단위 이행**
배송·정산·환불을 판매자별 SubOrder에서 처리합니다.

VII **결제 전 재고 선점**
결제 전에 재고를 선점해 주문이 몰려도 초과 판매를 막습니다.

VIII **멱등 결제**
멱등키로 재시도가 겹쳐도 결제가 한 번만 일어나게 합니다.

IX **배송완료 정산**
배송 완료분만 모아 수수료를 공제하고 판매자에게 지급합니다.

CONSTITUTION · 설계 해석

오늘의 설계를 좌우하는 원칙

constitution 2.0의 멀티셀러 원칙이 이후 모든 산출물의 판단 기준이 됩니다

SubOrder를 이행 단위로

다이어그램에서: 원칙 VI (배송·정산·환불이 SubOrder 위에서 동작)

결정

멀티셀러 주문을 판매자별 SubOrder로 나누고 이행 상태를 SubOrder에 둡니다.

근거

결제는 구매자당 1회지만 배송·정산·환불은 판매자별로 갈려야 합니다.

기각한 대안

주문을 단일 단위로 유지하면 판매자별 배송·정산을 표현할 수 없습니다.

결과

도메인·ERD·상태머신에서 모두 Order와 SubOrder를 나눠 설계합니다.

결제 전 재고 선점·멱등 결제

다이어그램에서: 원칙 VII·VIII (reserve 후 결제, idempotencyKey)

결정

주문 생성 시 재고를 선점하고 결제 authorize·void에 멱등키를 요구합니다.

근거

동시 주문의 오버셀과 네트워크 재시도의 중복 결제를 막아야 합니다.

기각한 대안

결제 후 재고를 차감하면 결제는 됐는데 재고가 없는 상태가 생깁니다.

결과

시퀀스의 재고 선점 후 결제 순서와 payments 테이블의 UQ 제약으로 이어집니다.

얇고 교체 가능한 Store 계승

다이어그램에서: 원칙 III·IX (Store 인터페이스, 배송 완료 후 정산)

결정

라우트·도메인은 Store 인터페이스에만 의존하고 정산은 배송 완료를 따릅니다.

근거

저장소 교체가 핸들러에 영향을 주지 않아야 하고 미배송 과지급을 막아야 합니다.

기각한 대안

핸들러가 구체 저장소에 직접 의존하면 교체·테스트가 어려워집니다.

결과

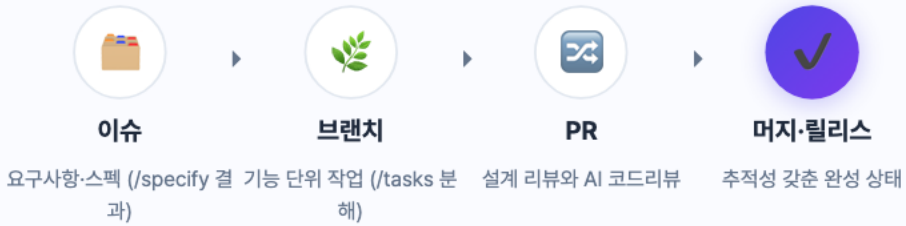
컴포넌트 아키텍처 3계층과 상태머신의 배송 완료 후 정산 트리거로 이어집니다.

constitution은 코드가 아니라 판단 기준입니다. 여기서 못 박은 아홉 원칙이 유스케이스·데이터·상태·구조 설계를 먼저 정합니다.

CONSTITUTION · GITHUB 협업 사이클

설계 산출물이 저장소에 남는 방식

스펙은 이슈로, 작업은 브랜치로, 리뷰는 PR로, 추적성은 머지로 남습니다



🔗 시니어 과정의 초점은 Git 기본기가 아니라 설계를 저장소에 그대로 남기는 협업 구조입니다.

CONSTITUTION · 체크포인트

블록 1 정리 (오프닝과 원칙)

- ✔ AI가 구현을 빠르게 할수록 사람은 설계 결정을 챙겨야 합니다.
- ✔ 멀티셀러 마켓플레이스 하나로 유스케이스부터 정산·환불까지 관통합니다.
- ✔ constitution 2.0의 아홉 원칙이 이후 모든 산출물의 판단 기준이 됩니다.
- ✔ 스펙은 이슈로, 작업은 브랜치로, 리뷰는 PR로, 추적성은 머지로 저장소에 남습니다.

SECTION

02 요구사항에서 멀티 액터 유스케이스로

- 먼저 유스케이스 다이어그램의 요소를 개념으로 익힙니다
- 그 요소로 그린 마켓플레이스 산출물을 함께 읽습니다
- 이 그림이 왜 이 형태인지 설계 결정을 해석합니다

50분 (블록 2) · /specify ①

SPECIFY ① · 유스케이스

Constitution 다음, 요구사항 시각화

방금 만든 것

constitution으로 시스템 경계와 원칙(얇은 Store 등)을 세웠습니다.

이제 할 것

유스케이스 다이어그램의 요소를 먼저 익히고 그 요소로 그린 산출물을 읽습니다.

다음으로 넘길 것

산출물의 형태를 해석해 다음 단계 도메인 모델의 클래스로 이어갑니다.

SPECIFY ① · 개념

유스케이스 다이어그램의 다섯 요소

누가(액터)·무엇을(유스케이스)·어디까지(시스템 경계)·필수(include)·조건부(extend)로 요구사항을 그립니다

액터
사람 모양

시스템 바깥에서 시스템을 사용하는 역할입니다. 실제 사람뿐 아니라 외부 시스템도 액터가 됩니다.

예시: 구매자·판매자·관리자, 그리고 결제 게이트웨이 같은 외부 시스템

유스케이스
타원

액터가 시스템을 통해 이루려는 기능입니다. 명사가 아니라 '무엇을 한다'는 동작으로 적습니다.

예시: 상품 검색·주문 생성·환불 요청

시스템 경계
큰 사각형

우리가 만드는 시스템의 책임 범위입니다. 사각형 안은 우리가 구현하고 밖은 연동만 합니다.

예시: 마켓플레이스 플랫폼 안 vs. 결제·정산 같은 외부 연동

include
필수 점선

한 유스케이스가 다른 유스케이스를 항상 포함합니다. 예외 없이 매번 함께 실행되는 관계입니다.

예시: 주문 생성은 결제 요청을 언제나 포함

extend
조건부 점선

특정 조건에서만 덧붙는 확장입니다. 정상 흐름에는 없다가 조건이 맞을 때만 일어납니다.

예시: 환불 요청은 분쟁일 때만 CS 중재로 확장

Ivar Jacobson, 'Object-Oriented Software Engineering', 1992 · UML 2.5 명세

SPECIFY ① · 실습

유스케이스 다이어그램을 만든 프롬프트

/speckit.specify로 만든 요구사항 스펙에 «include»·«extend» 관계까지 그려 달라고 요청하면 유스케이스 다이어그램이 나옵니다

실습

1 어느 단계에서

`/speckit.specify` 멀티셀러 요구사항을 spec.md 스펙 문서로 생성합니다

2 이렇게 요청합니다

스펙 생성 뒤 이어서 입력하는 지시

```
> 이 스펙의 액터와 기능을 멀티셀러 유스케이스 다이어그램으로, «include»·«extend» 관계까지 PlantUML로 그려줘
```

3 이런 산출물이 나옵니다

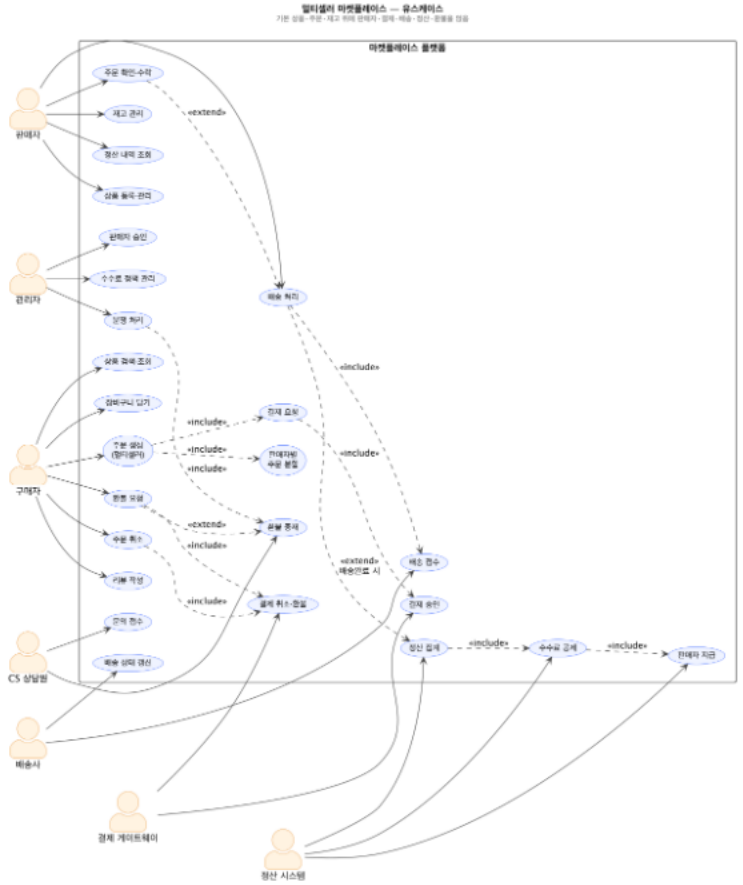
usecase-marketplace.puml
일곱 액터·스물여섯 유스케이스와 관계를 담은 PlantUML 소스

usecase-marketplace.png
렌더링한 다이어그램 이미지는 다음 산출물 장표에서 읽습니다

SPECIFY ① · 설명

멀티셀러 마켓플레이스 유스케이스

구매·판매·배송·정산·환불 요구사항을 그리니 일곱 액터와 스물여섯 유스케이스로 나왔습니다



타원 = 유스케이스(하는 일) · 사람 = 액터(역할) · 큰 사각형 = 시스템 경계 · 점선 화살표 = 유스케이스 사이 관계

다이어그램 읽기

만든 것 2편에서 만든 상품·주문·재고 위에 판매자·결제·배송·정산·환불을 얹어 멀티셀러로 확장한 요구사항입니다. 그 요구를 유스케이스로 옮기니 이 그림이 나왔습니다.

액터 7 좌측과 하단 바깥의 사람 아이콘이 액터입니다. 구매자·판매자·관리자·CS 상담원·배송사에 결제 게이트웨이·정산 시스템까지 일곱 역할이 참여합니다.

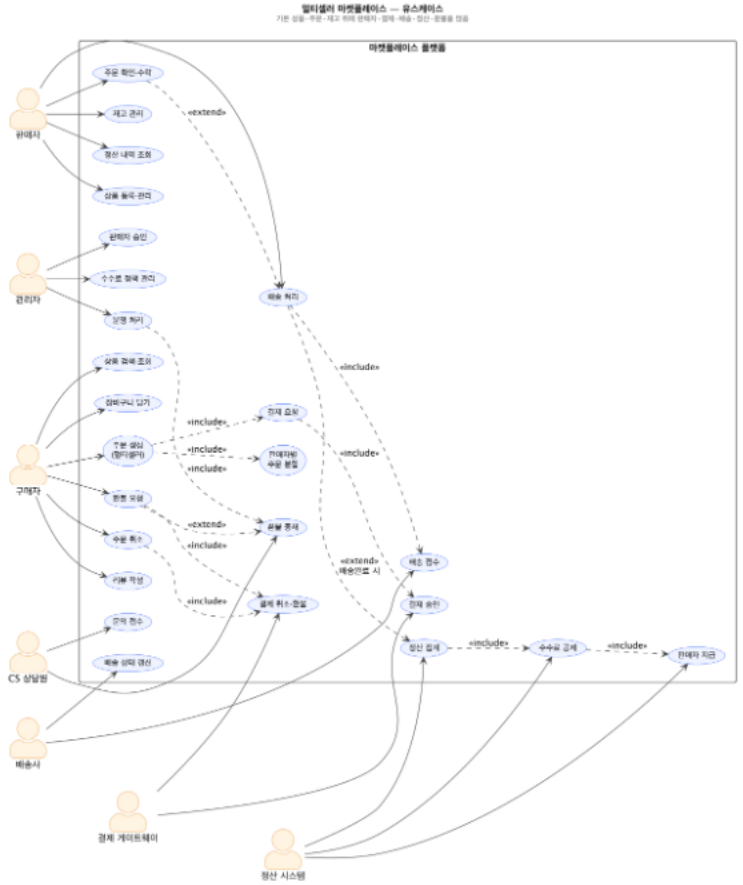
경계 화면을 크게 감싼 사각형이 마켓플레이스 플랫폼의 책임 범위입니다. 결제 게이트웨이·정산 시스템은 사각형 밖 외부 액터로 분리됩니다.

유스케이스 26 경계 안에 흩어진 타원 하나하나가 시스템이 제공하는 기능입니다. 상품 검색부터 정산 집계까지 스물여섯 개가 요구사항 여섯 묶음으로 모입니다.

SPECIFY ⓘ · 설명

이 그림 속 include와 extend

개념으로 배운 필수(include)·조건부(extend)가 실제로 어디에 쓰였는지 확인합니다



«include» = 항상 함께 실행되는 필수 하위 유스케이스 · «extend» = 특정 조건에서만 덧붙는 선택 유스케이스

다이어그램 읽기

include 가운데 주문 생성 타원에서 결제 요청과 판매자별 주문 분할로 «include» 점선이 뻗습니다. 멀티셀러 주문은 결제와 분할을 예외 없이 항상 포함합니다.

extend 배송 처리 타원에 «extend»(배송완료 시)로 정산 집계가 붙고 환불 요청에 «extend»로 환불 중재가 붙습니다. 조건이 맞을 때만 확장되는 흐름입니다.

정산 체인 우하단으로 이어지는 정산 집계·수수료 공제·판매자 지급이 정산 시스템입니다. 세 단계를 «include»로 이어 하나의 정산 절차로 묶습니다.

예외 분리 정상 경로(주문·결제·배송)와 예외 경로(분쟁 중재·정산 확장)를 한 그림에서 include와 extend로 갈라 둡니다. 무엇이 필수이고 무엇이 조건부인지 그림만 봐도 드러납니다.

SPECIFY ① · 개념

설계 결정 카드의 네 칸

이제부터 각 블록의 심화 장표에는 결정·근거·기각한 대안·결과를 남깁니다. 이렇게 판단의 근거를 남긴 기록을 ADR이라 합니다

결정

무엇을 하기로 했는지 한 문장으로 분명히 적습니다.

예시 · ADR-001
주문을 판매자별 SubOrder로 나눈다

근거

그 결정이 필요했던 문제와 제약을 밝힙니다.

예시 · ADR-001
한 주문에 여러 판매자가 섞인다

기각한 대안

택하지 않은 길과 버린 이유를 함께 남깁니다.

예시 · ADR-001
단일 주문 유지 (판매자별 배송 불가)

결과

이 결정으로 얻는 것과 감수하는 비용을 적습니다.

예시 · ADR-001
정산은 독립, 분할 로직은 필요

ADR(Architecture Decision Record)은 코드가 아니라 판단의 근거를 저장소에 남기는 짧은 기록입니다. 바로 다음 장부터 유스케이스가 미리 정하는 설계를 이 네 칸으로 해석합니다.

Michael Nygard, 'Documenting Architecture Decisions', 2011

SPECIFY ① · 심화

유스케이스가 미리 정하는 설계

그림의 각 형태가 이후 도메인·데이터 설계를 미리 결정합니다

결제·정산을 외부 액터로

다이어그램에서: 결제 게이트웨이·정산 시스템이 경계 밖 액터

결정

결제 승인과 정산 집계를 시스템 내부 기능이 아니라 외부 액터와의 상호작용으로 모델링했습니다.

근거

결제·정산은 재시도·멱등·정합성이 중요해 경계를 명확히 나눠야 합니다.

기각한 대안

~~결제를 내부 함수로 두기 (외부 실패·재시도 경계가 흐려집니다).~~

결과

결제 멱등키, 정산 배치 같은 경계 계약이 뒤 단계로 자연스럽게 이어집니다.

주문 분할을 필수(include)로

다이어그램에서: 주문 생성 ..> 판매자별 주문 분할 «include»

결정

주문 생성이 판매자별 주문 분할을 항상 포함하도록 include로 고정했습니다.

근거

멀티셀러에서는 한 주문에 여러 판매자가 섞이므로 분할이 예외가 아닌 기본입니다.

기각한 대안

~~단일 주문 유지 (판매자별 배송·정산·부분 환불을 표현할 수 없습니다).~~

결과

다음 도메인 모델의 Order/SubOrder 분리(ADR-001)로 곧장 이어집니다.

예외 경로를 extend로 분리

다이어그램에서: 배송완료 시 정산 확장 · 환불→중재 «extend»

결정

조건부로만 일어나는 흐름(배송 완료 시 정산, 환불 시 중재)을 extend로 떼어냈습니다.

근거

정상 경로와 예외 경로를 한 그림에서 구분해야 복잡성을 관리할 수 있습니다.

기각한 대안

~~모든 흐름을 실선으로 연결 (정상과 예외가 뒤섞여 읽기 어렵습니다).~~

결과

상태머신의 조건부 전이(배송완료→정산대상, 환불→역산)와 그대로 맞물립니다.

유스케이스는 그림이 아니라 결정의 묶음입니다. 여기서 include·extend로 못 박은 필수·예외 관계가 도메인·데이터·상태 설계를 먼저 정합니다.

SPECIFY ① · 체크포인트

블록 2 정리 (유스케이스)

- ✔ 액터·유스케이스·시스템 경계·include·extend 다섯 요소로 요구사항을 그림으로 옮깁니다.
- ✔ 그 요소로 멀티셀러 요구사항을 그리니 일곱 액터와 스물여섯 유스케이스가 나왔습니다.
- ✔ include는 필수 하위 흐름, extend는 조건부 확장으로 정상·예외 경로를 구분합니다.
- ✔ 결제·정산을 외부 액터로 분리한 결정이 뒤의 도메인·데이터·상태 설계를 미리 정합니다.

SECTION

03 도메인 개념 모델링

- 네 개의 바운디드 컨텍스트로 개념을 나눕니다
- Order와 SubOrder의 분할·집약 관계를 클래스로 그립니다
- 왜 주문을 SubOrder로 쪼갰는지 설계 근거를 해석합니다

50분 (블록 3) · /specify ②

SPECIFY ② · 도메인 모델

유스케이스 다음, 개념을 클래스로

방금 만든 것

유스케이스로 액터와 필수·선택 관계를 확정했습니다.

이제 할 것

개념을 바운디드 컨텍스트와 클래스·연관으로 구조화합니다.

다음으로 넘길 것

여기서 정한 Order/SubOrder 분리가 다음 단계 ERD의 테이블이 됩니다.

SPECIFY ② · 개념

도메인 클래스 모델의 다섯 요소

개념(클래스)·관계(연관)·강한 소유(합성 ◆)·느슨한 모음(집약 ◇)·책임 분리(바운디드 컨텍스트)로 도메인을 그립니다

클래스
개념 상자

속성과 메서드를 담은 개념입니다. 도메인에 등장하는 명사를 상자로 옮겨 책임을 붙입니다.

예시: Order·SubOrder·Inventory에 split()·reserve() 같은 메서드가 붙습니다

연관
실선

두 클래스를 실선으로 잇는 관계입니다. 서로를 알고 참조하지만 소유하지는 않습니다.

예시: Shipment가 SubOrder에 0..1로 붙어 배송을 참조합니다

합성
채운 다이아몬드 ◆

채운 다이아몬드로 표기하는 강한 소유입니다. 부분은 전체 없이 존재하지 못하고 함께 사라집니다.

예시: Order ◆ SubOrder — SubOrder는 Order 없이 존재하지 못합니다

집약
빈 다이아몬드 ◇

빈 다이아몬드로 표기하는 느슨한 모음입니다. 부분은 독립적으로 존재하며 여러 곳에 모일 수 있습니다.

예시: Settlement ◇ SubOrder — 배송 완료된 것만 느슨하게 모아 정산합니다

바운디드 컨텍스트
영역(패키지)

관련 클래스를 하나의 영역으로 묶어 책임을 분리하는 경계입니다. 경계 안에서만 개념의 의미가 통합니다.

예시: 판매자·주문·결제/정산·배송 네 영역으로 도메인을 나눕니다

UML 2.5 명세 · Eric Evans, 'Domain-Driven Design', 2003

SPECIFY ② · 실습

도메인 클래스 모델을 만든 프롬프트

/speckit.plan의 Data Model 단계에서 합성·집약·바운디드 컨텍스트를 구분해 달라고 요청하면 도메인 모델이 나옵니다

실습

1 어느 단계에서

`/speckit.plan` 스펙을 근거로 설계 계획(plan.md)의 Data Model 절을 작성합니다

2 이렇게 요청합니다

plan의 Data Model 단계에서 입력하는 지시

```
> 이 스펙을 도메인 클래스 모델로, 합성(◆)·집약(◇)·바운디드 컨텍스트를 구분해 PlantUML로 그려줘
```

3 이런 산출물이 나옵니다

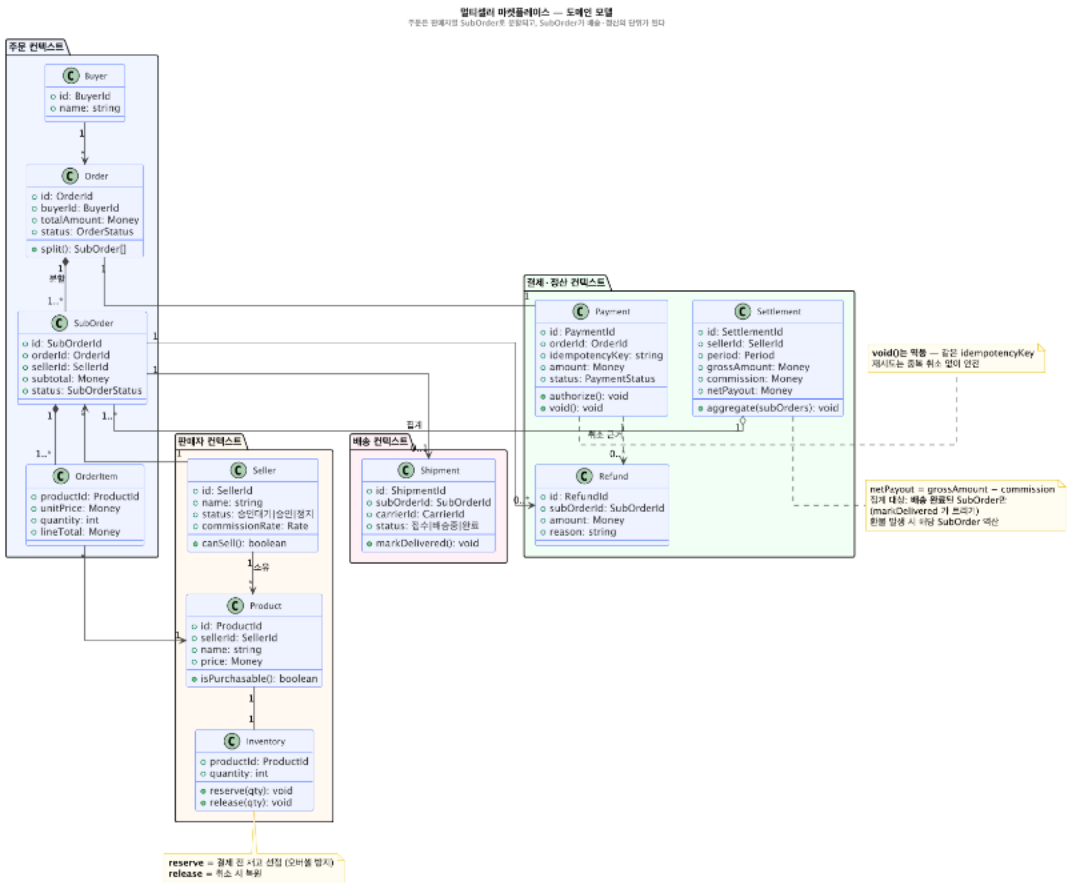
domain-model.puml
클래스·합성·집약·바운디드 컨텍스트를 담은 PlantUML 소스

plan.md · Data Model
도메인 모델의 근거가 되는 계획 문서 절

SPECIFY ② · 설명

멀티셀러 도메인 클래스 모델

도메인 개념을 클래스로 그리니 바운디드 컨텍스트 네 개와 Order·SubOrder 분할이 나왔습니다



클래스 상자 = 개념(속성·메서드) · 패키지 = 바운디드 컨텍스트 · 실선 화살표 = 연관 · 다이아몬드 = 집약/합성

다이어그램 읽기

만든 것 도메인 개념을 클래스로 그리니 바운디드 컨텍스트 4개와 Order/SubOrder 분할이 나왔습니다. 앞 장의 다섯 요소를 실제 상자와 선으로 배치한 결과입니다.

주문 왼쪽에 세로로 놓인 Buyer·Order·SubOrder·OrderItem이 주문 컨텍스트입니다. Order가 split()으로 SubOrder를 만듭니다.

판매자 그 아래 판매자 컨텍스트에 Seller·Product·Inventory가 묶입니다. Inventory에 reserve·release 메서드가 있습니다.

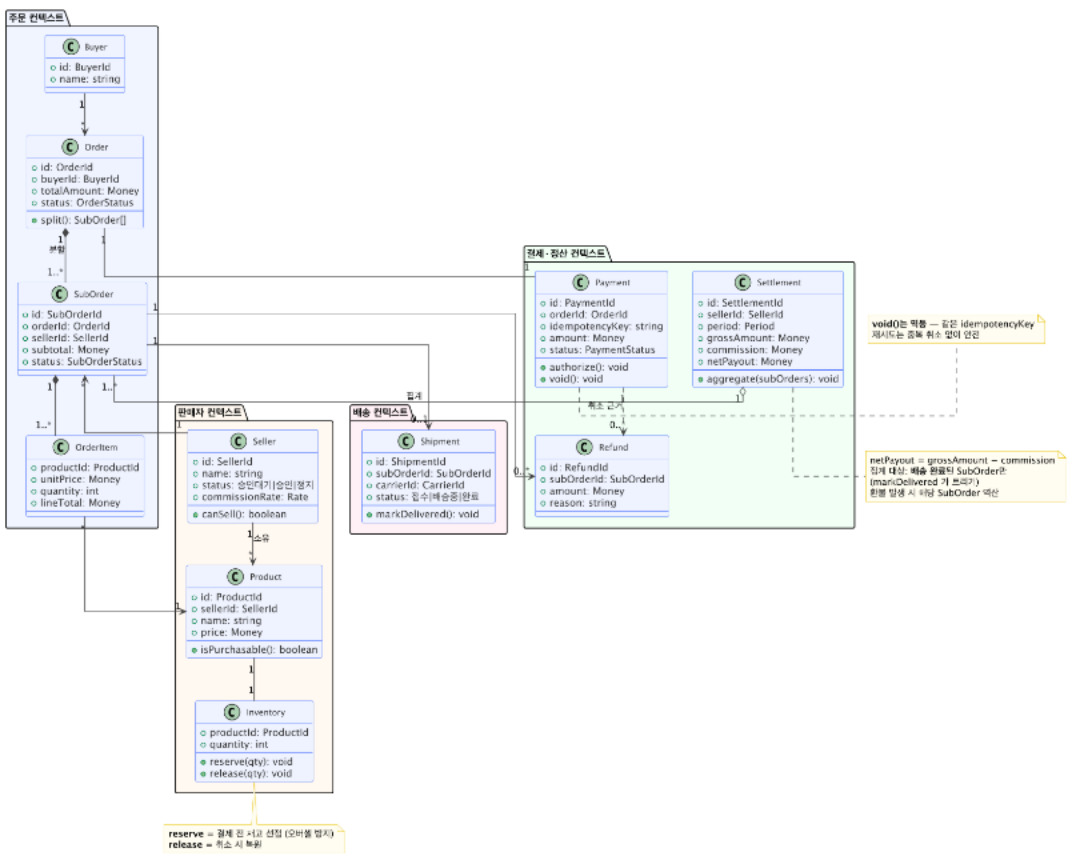
결제·정산 오른쪽 위 패키지에 Payment·Settlement·Refund가 모입니다. Settlement가 배송 완료된 SubOrder를 집계합니다.

배송 가운데 배송 컨텍스트의 Shipment는 SubOrder에 0..1로 붙습니다. markDelivered가 정산의 트리거가 됩니다.

SPECIFY ② · 설명

Order·SubOrder 분할과 정산 집약

개념으로 배운 합성(◆)·집약(◇)이 실제로 어디에 쓰였는지 표기법으로 확인합니다



채운 다이아몬드(◆) = 합성(강한 소유) · 빈 다이아몬드(◇) = 집약(느슨한 모음) · 1..* = 다중도

다이어그램 읽기

합성 좌측 Order와 SubOrder를 잇는 채운 다이아몬드가 합성입니다. Order가 SubOrder를 1..*로 소유하며 SubOrder는 Order 없이 존재하지 않습니다.

분할 그 합성선을 따라 Order.split()이 판매자별 SubOrder 배열을 만듭니다. 멀티셀러에서 분할은 기본 동작입니다.

집약 우측 Settlement에서 SubOrder로 향한 빈 다이아몬드가 집약입니다. 배송 완료된 것만 느슨하게 모아 지급합니다.

결제 우측 Payment는 Order에 1대1로 붙습니다. 결제는 주문 단위(1회)지만 이행 상태는 SubOrder 단위입니다.

SPECIFY ② · 심화

도메인 모델이 가르는 경계

클래스 경계와 메서드가 이후 데이터·상태 설계를 미리 정합니다

ADR-001 Order와 SubOrder를 분리

다이어그램에서: Order 1 ◆-- 1.. SubOrder (합성)*

결정

주문을 판매자별 SubOrder로 분할하고 배송·정산·환불의 단위를 SubOrder로 둡니다.

근거

한 주문에 여러 판매자가 섞이므로, 정산과 부분 환불이 판매자별로 독립해야 합니다.

기각한 대안

주문을 단일 단위로 유지하면 판매자별 정산·부분 배송·부분 환불을 표현할 수 없습니다.

결과

ERD의 orders·sub_orders 분리, 상태머신의 SubOrder 라이프사이클로 곧장 이어집니다.

ADR-002 재고에 reserve·release 명시

다이어그램에서: Inventory.reserve(qty) · release(qty)

결정

재고를 결제 전에 선점(reserve)하고 취소·실패 시 복원(release)하도록 메서드를 둡니다.

근거

동시 주문 시 같은 재고를 여러 명이 결제하는 오버셀을 막아야 합니다.

기각한 대안

결제됐지만 재고가 없는 상태가 발생합니다.

결과

결제 실패 시 롤백(release) 경로가 필요해지고 시퀀스에서 선점 흐름으로 드러납니다.

ADR-003 결제 취소를 멱등으로

다이어그램에서: Payment.void()는 멱등 · idempotencyKey

결정

Payment에 idempotencyKey를 두고 같은 키의 재요청을 중복 처리하지 않습니다.

근거

네트워크 재시도·중복 클릭에도 중복 결제나 중복 취소가 나면 안 됩니다.

기각한 대안

매 요청을 그대로 실행하면 재시도가 중복 결제·중복 환불을 일으킵니다.

결과

ERD의 payments.idempotency_key UQ 제약으로 데이터 수준에서 강제됩니다.

도메인 모델의 클래스 경계는 그림이 아니라 결정입니다. Order/SubOrder를 나눈 순간, 데이터·상태·정산 설계의 형태가 이미 정해집니다.

SPECIFY ② · 체크포인트

블록 3 정리 (도메인 모델)

- ✔ 클래스·연관·합성·집약·바운디드 컨텍스트 다섯 요소를 먼저 익혀 개념을 클래스로 옮깁니다.
- ✔ 네 개 바운디드 컨텍스트로 판매자·주문·결제·정산·배송의 책임을 갈랐습니다.
- ✔ Order를 SubOrder로 합성 분할해 배송·정산·환불의 단위를 SubOrder로 두었습니다(ADR-001).
- ✔ reserve·release와 멱등 void를 도메인에 명시해 오버셀·중복 결제를 막습니다.
- ✔ 다음 블록에서 이 개념을 테이블·키·제약으로 옮긴 ERD를 읽습니다.

SECTION

04 데이터 설계

- 개념 모델을 테이블·키·관계로 옮깁니다
- orders와 sub_orders 분리가 멀티셀러 정산의 토대임을 확인합니다
- UQ 제약과 브릿지 테이블이 어떤 결정의 결과인지 해석합니다

50분 (블록 4) · /plan ①

PLAN ① · ERD

도메인 개념을 테이블로 옮깁니다

방금 만든 것

바운디드 컨텍스트와 Order/SubOrder 분리를 클래스로 확정했습니다.

이제 할 것

개념을 테이블·기본키·외래키·제약으로 옮겨 데이터 구조를 잡습니다.

다음으로 넘길 것

여기서 잡은 정적 구조 위로 다음 단계에서 동적 흐름과 상태를 얹습니다.

PLAN ① · 개념

ERD의 다섯 요소

엔티티(테이블)·PK(기본키)·FK(외래키)·UQ(유니크)·카디널리티로 데이터 구조를 그립니다

엔티티 **테이블**	데이터를 담는 표입니다. 한 종류의 대상을 행으로 쌓고 속성을 열로 나눠 저장합니다. **예시:** orders(주문 헤더)·sub_orders(판매자별 하위주문)·payments(결제)
기본키 **«PK»**	한 행을 유일하게 식별하는 키입니다. 같은 값이 두 번 나올 수 없고 비어 있을 수도 없습니다. **예시:** orders.id — 주문 하나를 가리키는 고유 식별자
외래키 **«FK»**	다른 테이블의 행을 가리키는 참조입니다. 두 테이블을 연결해 관계를 만듭니다. **예시:** sub_orders.order_id — 어느 orders에 속하는지 참조
유니크 **«UQ»**	한 열에 같은 값이 두 번 들어가지 못하게 막는 중복 금지 제약입니다. **예시:** payments.idempotency_key — 같은 키의 중복 결제 차단
카디널리티 **까치발** ⋔	두 테이블이 몇 대 몇으로 이어지는지 나타냅니다. 까치발(crow's foot)이 다(N) 쪽을 가리킵니다. **예시:** orders 1 : N sub_orders, 정산과 하위주문은 N : M

Peter Chen, 'The Entity-Relationship Model', 1976 · SQL 표준

PLAN ① · 실습

관계형 스키마 ERD를 만든 프롬프트

/speckit.plan의 data-model 산출물을 관계형 스키마로, PK·FK·UQ와 카디널리티를 표기해 달라고 요청하면 ERD가 나옵니다

실습

1 어느 단계에서

`/speckit.plan` 앞서 만든 도메인 모델(data-model)을 관계형 스키마로 변환합니다

2 이렇게 요청합니다

data-model을 바탕으로 입력하는 지시

```
> 도메인 모델을 관계형 스키마 ERD로, PK·FK·UQ와 카디널리티를 표기해 PlantUML로 그려줘
```

3 이런 산출물이 나옵니다

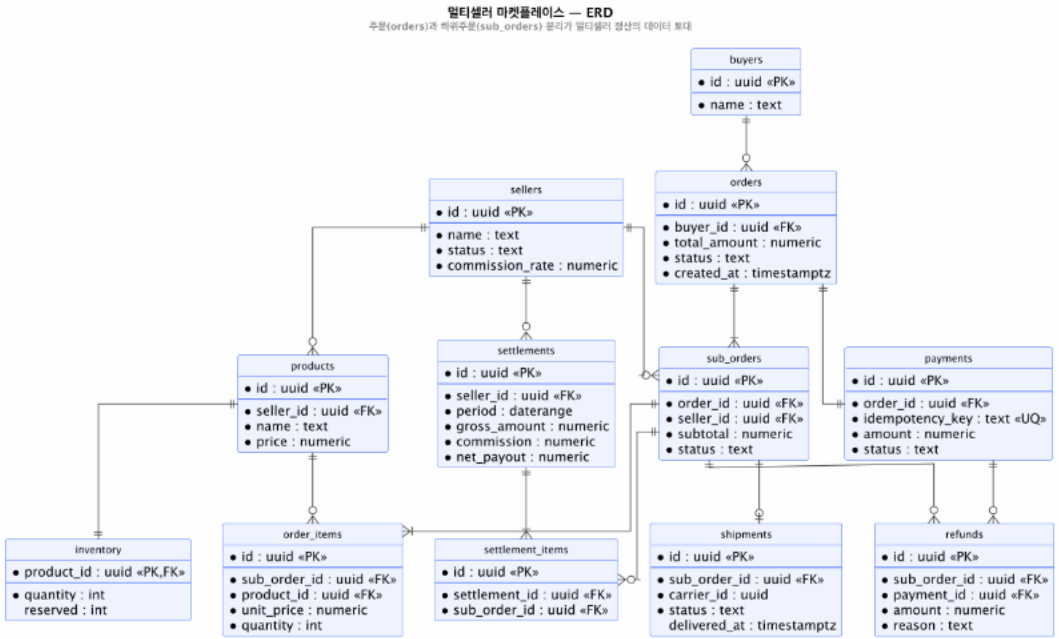
erd.puml
테이블·PK·FK·UQ·카디널리티를 담은 PlantUML 소스

erd.png
렌더링한 ERD 이미지는 다음 산출물 장표에서 읽습니다

PLAN ① · 설명

멀티셀러 데이터 모델 ERD

주문(orders)과 하위주문(sub_orders) 분리가 데이터 토대임을 확인합니다



«PK» = 기본키 · «FK» = 외래키 · «UQ» = 유니크 제약 · 까치발(⋔) = 다(N) 쪽 · 원(○) = 선택(0)

다이어그램 읽기

만든 것

도메인 개념을 옮기면 12개 테이블과 orders/sub_orders로 나뉩니다. 앞 장의 엔티티·PK·FK·UQ·카디널리티가 여기에 실제로 배치됩니다.

orders

가운데 위쪽 orders가 주문 헤더입니다. buyer_id(FK)·total_amount·status를 갖고 결제는 이 단위로 한 번 일어납니다.

sub_orders

orders 바로 아래 sub_orders가 1대다로 연결됩니다. order_id·seller_id(FK)·subtotal·status로 판매자별 이행을 담습니다.

payments

우측 payments가 orders에 1대1로 붙습니다. idempotency_key의 «UQ»가 데이터 수준에서 중복 결제를 막습니다.

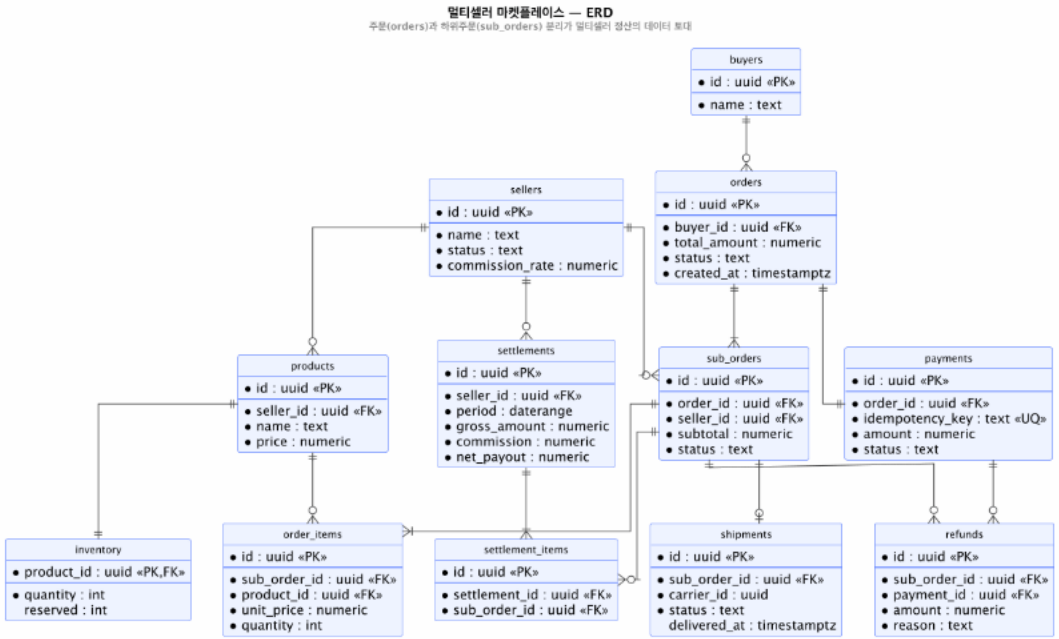
settlements

좌측 settlements가 판매자별 정산 헤더입니다. gross_amount·commission·net_payout으로 지급액을 계산합니다.

PLAN ⓘ · 설명

정산 브릿지 테이블과 역방향 관계

정산과 하위주문의 다대다를 브릿지 테이블로 풀어낸 부분을 봅니다



브릿지(연결) 테이블 = 다대다를 두 개의 1대다로 분해 · settlement_items가 정산과 SubOrder를 잇습니다

다이어그램 읽기

브릿지

좌측 settlements와 가운데 sub_orders 사이에 놓인 settlement_items가 브릿지입니다. settlement_id·sub_order_id 두 FK만 가진 연결 테이블입니다.

다대다

그 브릿지에서 양쪽으로 뻗은 까치발이 다대다를 나타냅니다. 한 정산은 여러 SubOrder를 담고 한 SubOrder는 여러 정산에 걸립니다.

역산

환불이 나면 기록된 settlement_items를 근거로 차기 정산에서 역산(차감)합니다.

분리

sub_orders가 orders에서 떨어진 별도 테이블이라 판매자별 정산·배송·환불을 각각 조인해 집계할 수 있습니다.

PLAN ① · 심화

데이터 구조에 새긴 결정

테이블별 제약이 애플리케이션 로직을 대신합니다

ADR-003 멱등키를 UQ 제약으로

다이어그램에서: payments.idempotency_key : text «UQ»

결정

결제 멱등키를 애플리케이션 검증이 아니라 테이블 UQ 제약으로 강제합니다.

근거

코드가 놓쳐도 데이터베이스가 같은 키의 두 번째 결제를 거부합니다.

기각한 대안

~~코드에서만 중복을 검사하면 동시 요청이 검사를 통과해 중복 결제가 생길 수 있습니다.~~

결과

데이터베이스는 재시도·중복 클릭에도 결제를 한 번만 기록합니다.

ADR-001 orders와 sub_orders 분리

다이어그램에서: orders ||--|{ sub_orders (1대다)

결정

주문 헤더(orders)와 판매자별 하위주문(sub_orders)을 별도 테이블로 나눕니다.

근거

배송·정산·환불이 판매자별로 갈리므로, 상태를 sub_orders에 둬야 조인이 깔끔합니다.

기각한 대안

~~주문 한 테이블에 판매자 컬럼을 추가하면 한 주문에 여러 판매자를 담을 수 없습니다.~~

결과

정산·배송·환불 테이블이 모두 sub_order_id를 FK로 참조하는 구조가 됩니다.

정산을 브릿지 테이블로

다이어그램에서: settlement_items (settlement_id + sub_order_id)

결정

정산과 하위주문의 다대다를 settlement_items 브릿지로 분해합니다.

근거

어느 정산이 어느 SubOrder를 담았는지 명시해야 환불 시 역산 근거가 남습니다.

기각한 대안

~~정산에 SubOrder 목록을 JSON으로 저장하면 조인·집계·역산이 어려워집니다.~~

결과

차기 정산에서 이미 정산된 건을 정확히 찾아 차감할 수 있습니다(ADR-004 역산).

ERD의 제약은 문서가 아니라 실행되는 규칙입니다. UQ·FK·분리가 코드보다 먼저 데이터의 정합성을 지킵니다.

PLAN ① · 체크포인트

블록 4 정리 (ERD)

- ✔ 엔티티·PK·FK·UQ·카디널리티 다섯 요소를 먼저 익히고 도메인을 테이블로 옮겼습니다.
- ✔ orders와 sub_orders 분리가 판매자별 배송·정산·환불의 데이터 토대입니다.
- ✔ payments.idempotency_key의 UQ 제약이 데이터 수준에서 중복 결제를 막습니다.
- ✔ settlement_items 브릿지가 다대다를 풀고 환불 역산의 근거를 남깁니다.
- ✔ 다음 블록에서 이 정적 구조에 시퀀스와 상태머신의 동적 흐름을 얹습니다.

SECTION

05 상호작용과 상태

- 주문에서 정산까지의 동적 흐름을 시퀀스로 읽습니다
- SubOrder의 상태 전이를 상태머신으로 읽습니다
- 결제 전 재고 선점과 배송 완료 정산 트리거의 근거를 해석합니다

50분 (블록 5) · /plan ②

PLAN ② · 시퀀스·상태머신

정적 구조 위에 동적 흐름을 얹습니다

방금 만든 것

ERD로 테이블·키·관계의 정적 구조를 확정했습니다.

이제 할 것

주문에서 정산까지의 흐름(시퀀스)과 SubOrder 상태 전이(상태머신)를 읽습니다.

다음으로 넘길 것

여기서 정한 흐름·상태가 다음 단계 컴포넌트 구조와 ADR로 정리됩니다.

PLAN ② · 개념

시퀀스와 상태머신의 표기 요소

시퀀스는 참여자 사이의 호출 순서를, 상태머신은 한 객체의 상태 전이를 그립니다

시퀀스

참여자·라이프라인

상단의 참여자(participant)에서 아래로 내려가는 세로줄이 라이프라인(생명선)입니다. 위에서 아래로 시간이 흐릅니다.

예시: 주문 서비스·재고·결제 게이트웨이가 참여자, 각자의 세로줄이 라이프라인

시퀀스

메시지 화살표

참여자 사이를 잇는 화살표입니다. 실선은 호출(요청), 점선은 응답(반환)이며, 라이프라인 위 막대가 활성화 구간입니다.

예시: reserve → authorize 호출(실선)과 승인 결과 반환(점선)

시퀀스

alt·group

여러 메시지를 감싸는 상자입니다. group은 하나로 묶고, alt는 조건에 따른 분기(성공/실패)를 나눕니다.

예시: 재고 선점 묶음(group), 재고 부족 시 실패로 갈리는 분기(alt)

상태머신

상태·전이

둥근 상자가 상태(state), 상자를 잇는 화살표가 전이(transition)입니다. 전이 라벨이 전이를 일으키는 사건입니다.

예시: 생성됨·결제대기·배송완료 상태와 그 사이를 잇는 전이

상태머신

가드 [조건]

전이 라벨의 대괄호가 가드(조건)이며, 함께 일어나는 부수효과(액션)를 표기합니다. 조건이 맞아야 전이가 일어납니다.

예시: 배송 전 취소 시 [결제 void + 재고 release]를 함께 수행

상태머신

시작 ● · 종료 ⦿

채운 원(●)이 시작 지점, 테두리 두른 원(⦿)이 종료 지점입니다. 생애주기가 어디서 시작해 어디서 끝나는지 표시합니다.

예시: 주문 생성이 시작(●), 정산완료·환불종료가 종료(⦿)

PLAN ② · 실습

시퀀스·상태머신을 만든 프롬프트

/speckit.plan에서 주문부터 정산까지 흐름과 SubOrder 생애주기를 함께 그려 달라고 요청하면 두 다이어그램이 나옵니다

실습

1 어느 단계에서

`/speckit.plan` 주문·정산 상호작용과 하위주문 상태 전이를 함께 계획합니다

2 이렇게 요청합니다

상호작용과 상태를 한 번에 요청하는 지시

```
> 주문에서 정산까지 흐름을 시퀀스 다이어그램으로, SubOrder 생애주기를 상태머신으로(가드·전이 포함) PlantUML로 그려줘
```

3 이런 산출물이 나옵니다

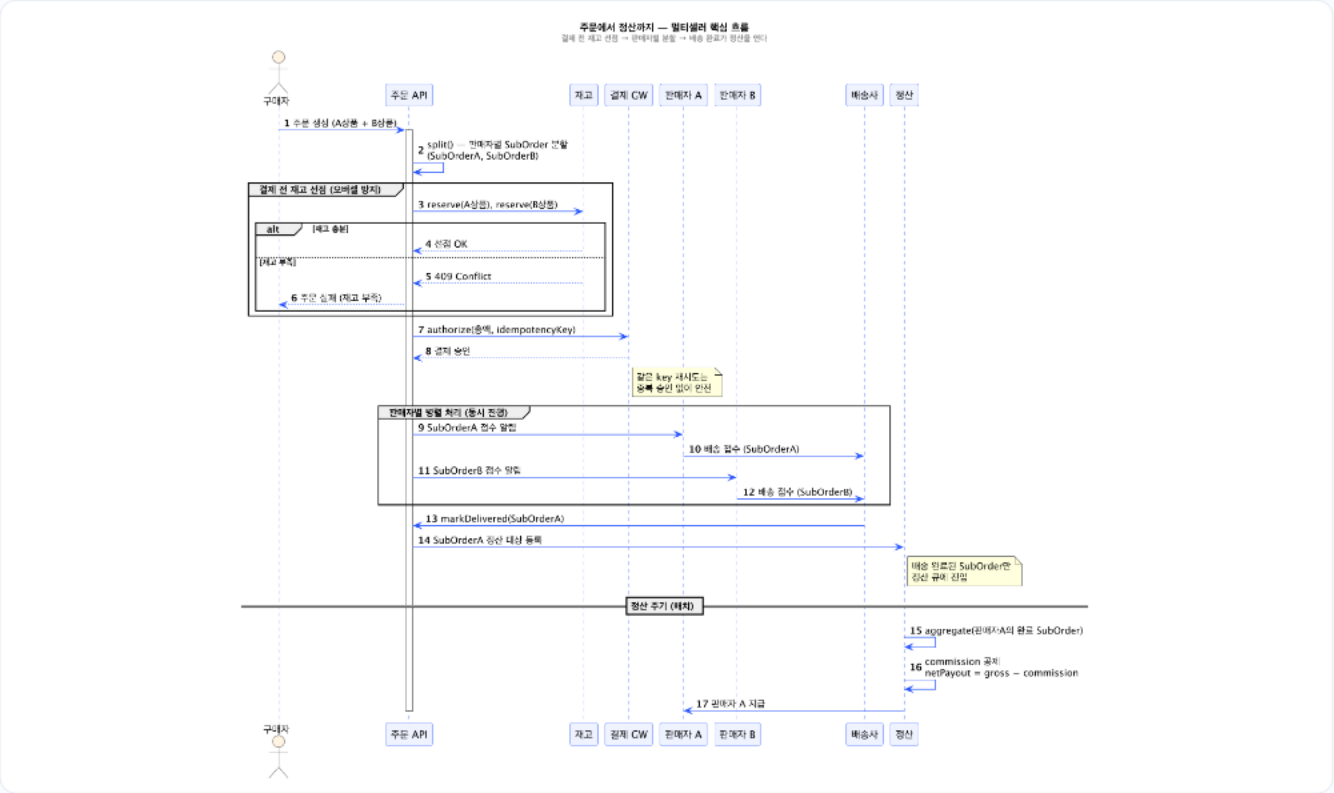
sequence-order-to-settlement.puml
주문·결제·배송·정산 상호작용을 담은 시퀀스

state-suborder.puml
SubOrder 상태와 가드·전이를 담은 상태머신

PLAN ② · 설명

주문에서 정산까지의 시퀀스

분할·선점·결제·배송완료·정산의 호출 순서를 시퀀스로 옮긴 결과물입니다



세로선 = 생명선(참여자) · 실선 화살표 = 호출 · 점선 = 응답 · group/alt = 묶음과 분기

다이어그램 읽기

만든 것 주문 생성부터 정산까지의 흐름을 시퀀스로 그리니 분할→선점→결제→배송완료→정산 순서가 나왔습니다.

분할 위에서 아래로 시간이 흐릅니다. 최상단 주문 생성 직후 split() 호출로 판매자별 SubOrder(A·B)를 나눕니다. 이후 흐름은 SubOrder 단위입니다.

선점 그 아래 결제보다 먼저 reserve로 재고를 선점하는 묶음이 있습니다. 부족하면 409로 실패하고 아무것도 만들지 않습니다.

결제 가운데쯤 authorize 호출에 idempotencyKey를 실어 승인합니다. 같은 키 재시도는 중복 승인 없이 안전합니다.

정산 우하단 배치 흐름이 마지막입니다. 배송 완료된 SubOrder만 정산 큐에 들어가고, 집계·수수료 공제 후 지급합니다.

PLAN ② · 심화

호출 순서가 막아내는 것

호출 순서 하나가 오버셀·중복 결제·과지급을 막는 장치입니다

ADR-002 결제 전에 재고를 선점

다이어그램에서: reserve → authorize 순서 (선점이 결제보다 먼저)

결정

주문 생성 시 재고를 먼저 선점하고, 성공하면 결제를 승인합니다.

근거

동시 주문이 같은 재고를 결제하는 오버셀을 원천 차단해야 합니다.

기각한 대안

~~결제 후 재고 차감(결제는 됐는데 재고가 없는 상태가 생깁니다).~~

결과

대신 결제 실패 시 release 롤백 경로가 필요합니다.

ADR-003 결제를 멱등 연산으로

다이어그램에서: authorize(총액, idempotencyKey) + 재시도 안전 노트

결정

authorize·void에 멱등키를 실어 같은 키로 다시 요청해도 중복 처리하지 않습니다.

근거

네트워크 재시도·중복 클릭에도 이중 결제·이중 취소가 나면 안 됩니다.

기각한 대안

~~매 요청 그대로 실행(재시도가 중복 결제를 일으킵니다).~~

결과

같은 키로 다시 요청하면 원래 결과를 돌려주어 금전 연산을 안전하게 처리합니다.

ADR-004 배송 완료가 정산을 연다

다이어그램에서: markDelivered → 정산 대상 등록 → 배치 집계

결정

SubOrder가 배송 완료돼야 정산 큐에 들어가고, 주기 배치에서 집계·지급합니다.

근거

미배송 건을 정산하면 과지급이 생기므로, 완료를 트리거로 삼습니다.

기각한 대안

~~결제 즉시 정산(배송 실패·반품 시 이미 지급한 돈을 회수해야 합니다).~~

결과

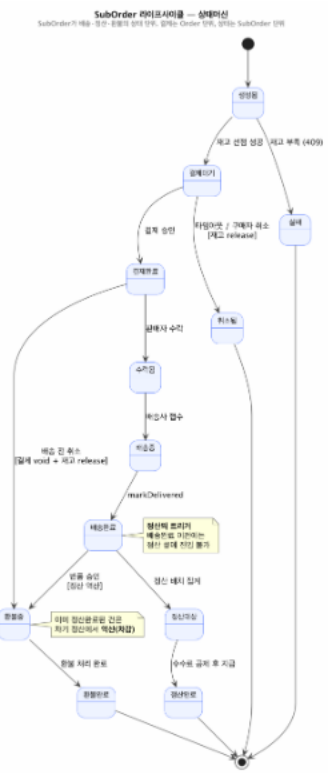
환불이 나면 이미 정산된 건을 차기 정산에서 역산(차감)합니다.

시퀀스는 단순한 순서도가 아닙니다. reserve→authorize→배송완료→정산의 순서 자체가 정합성을 지키는 설계 결정입니다.

PLAN ② · 설명

SubOrder 생애주기의 상태머신

생성부터 정산·환불 종료까지, 각 전이의 조건과 부수효과를 담은 결과물입니다



둥근 상자 = 상태 · 화살표 = 전이 · 라벨 = 전이 조건 · [대괄호] = 함께 일어나는 부수효과(가드)

다이어그램 읽기

만든 것 SubOrder의 생애주기를 상태머신으로 그리니 생성→결제→배송→정산→환불 전이가 나왔습니다.

선점 위에서 아래로 상태가 흐릅니다. 최상단 생성됨에서 재고 선점에 성공하면 결제대기, 부족하면 실패(409)로 갈립니다.

가드 전이 화살표 라벨의 [대괄호]가 부수효과입니다. 배송 전 취소는 [결제 void + 재고 release]를 함께 수행합니다.

트리거 중간 아래쪽 배송완료가 정산의 트리거입니다. 이 상태를 지나기 전에는 정산 큐에 진입할 수 없습니다.

역산 하단 정산완료 상태에서 환불이 나면 차기 정산에서 역산(차감)합니다.

PLAN ② · 심화

상태 전이가 정하는 정합성

상태를 어디에 두고, 어떤 전이를 허용하느냐가 정합성을 결정합니다

ADR-001 상태를 SubOrder에 둔다

다이어그램에서: 상태머신의 주어가 Order가 아니라 SubOrder

결정

결제는 Order 단위지만, 배송·정산·환불 상태는 SubOrder마다 독립하게 둡니다.

근거

판매자 A는 배송완료인데 B는 배송중일 수 있어, 상태가 판매자별로 갈려야 합니다.

기각한 대안

~~Order 하나에 상태를 둠(판매자별 부분 배송·부분 정산을 표현할 수 없습니다).~~

결과

한 주문 안에서 SubOrder들이 서로 다른 상태로 진행할 수 있습니다.

ADR-004 배송완료를 정산 트리거로

다이어그램에서: 배송완료 → 정산대상 (정산 배치 집계)

결정

SubOrder가 배송완료 상태일 때만 정산대상으로 전이할 수 있습니다.

근거

미배송 건을 정산하면 과지급이 생기므로, 배송 완료를 조건으로 둡니다.

기각한 대안

~~결제완료 즉시 정산(배송 실패 시 이미 지급한 돈을 회수해야 합니다).~~

결과

상태머신에 정산 정합성 규칙을 담습니다. 완료 전에는 지급할 수 없습니다.

환불을 역방향 전이로

다이어그램에서: 정산완료 → 환불중 [정산 역산]

결정

환불을 별도 흐름이 아니라 기존 상태에서 환불중으로 가는 역방향 전이로 그립니다.

근거

정상 경로와 예외 경로를 한 상태머신에서 봐야 누락 없이 관리됩니다.

기각한 대안

~~환불을 별도 다이어그램으로 분리(어느 상태에서 환불이 가능한지 흐려집니다).~~

결과

이미 정산완료된 건의 환불은 차기 정산에서 차감하도록 연결합니다.

상태머신은 허용된 전이의 목록이자 금지된 전이의 목록입니다. 배송 없이 정산으로 갈 수 없다는 규칙이 그림 안에 담겨 있습니다.

PLAN ② · 정합

같은 사실을 보는 시퀀스와 상태머신

흐름·상태 다이어그램의 상호 검증

01 재고 선점

시퀀스의 reserve 호출은 상태머신의 생성됨→결제대기 전이와 같은 사건입니다.

02 결제 승인

시퀀스의 authorize 응답은 상태머신의 결제대기→결제완료 전이와 같습니다.

03 배송 완료

시퀀스의 markDelivered는 상태머신의 배송중→배송완료 전이와 같은 순간입니다.

04 정산 트리거

시퀀스의 정산 대상 등록은 상태머신의 배송완료→정산대상 전이가 됩니다.

🔍 두 다이어그램이 어긋나면 설계에 구멍이 있다는 신호입니다. 흐름과 상태가 일치해야 구현이 안전합니다.

PLAN ② · 체크포인트

블록 5 정리 (시퀀스와 상태머신)

- ✔ 두 다이어그램의 표기 요소인 참여자·메시지·전이·가드를 먼저 익혔습니다.
- ✔ reserve→authorize 순서가 오버셀을, 멱등키가 중복 결제를 막습니다.
- ✔ 상태를 SubOrder에 둬 판매자별 부분 배송·부분 정산을 표현합니다(ADR-001).
- ✔ 배송완료가 정산의 트리거이며, 그 이전에는 정산 큐에 진입할 수 없습니다(ADR-004).
- ✔ 시퀀스와 상태머신이 같은 사건을 다른 각도로 담아 서로를 검증합니다.

SECTION

06 구조와 아키텍처 결정

- 라우트-도메인-Store 3계층 구조를 읽습니다
- ADR이 무엇이고 왜 남기는지 정리합니다
- 다섯 개 ADR을 유스케이스·시퀀스·상태와 연결해 종합합니다

50분 (블록 6) · /plan ③

PLAN ③ · 컴포넌트·ADR

흐름과 상태 다음의 코드 구조와 결정

방금 만든 것

시퀀스와 상태머신으로 동적 흐름과 상태 전이를 확정했습니다.

이제 할 것

코드가 어떤 계층으로 나뉘는지 보고 그 결정을 ADR로 남깁니다.

다음으로 넘길 것

여기서 정리한 결정이 다음 단계 작업 분해와 추적성 매트릭스의 기준이 됩니다.

PLAN ③ · 개념

컴포넌트 다이어그램의 다섯 요소

C4 네 수준(Context·Container·Component·Code) 중 오늘 쓰는 세 수준에서 상자(컴포넌트)·화살표(의존)·계층·외부 시스템으로 코드 구조를 그립니다

C4 4레벨

Context·Container·Component

C4는 시스템을 Context·Container·Component·Code 네 수준으로 봅니다. 오늘은 Code를 뺀 세 수준에서 시스템 경계부터 배포 단위(컨테이너), 코드 모듈(컴포넌트)로 좁혀 갑니다.

> **예시:** 마켓플레이스 시스템 > 백엔드 컨테이너 > 라우트·도메인·Store 컴포넌트

컴포넌트

상자

하나의 코드 모듈입니다. 한 가지 책임을 맡는 구현 단위를 상자 하나로 그립니다.

> **예시:** Hono 라우트, 도메인 서비스, Store 인터페이스

의존 방향

화살표

한 모듈이 다른 모듈을 사용한다는 뜻입니다. 화살표가 가리키는 쪽에 의존하므로 화살표 방향이 결합 방향입니다.

> **예시:** 라우트 → 도메인 서비스 → Store (아래로 내려가는 단방향)

계층 분리

가로 띠

역할이 같은 모듈을 한 층으로 묶습니다. 진입·로직·저장을 층으로 갈라 의존이 한 방향으로만 흐르게 합니다.

> **예시:** 라우트 계층 · 도메인 계층 · 저장소 계층의 세 층

외부 시스템

구름

우리가 만들지 않고 연동만 하는 대상입니다. 경계 밖 구름으로 그려 책임 범위를 분명히 합니다.

> **예시:** 결제 게이트웨이, 배송사 API 같은 외부 연동 대상

Simon Brown, 'The C4 Model for Software Architecture', 2018

PLAN ③ · 실습

컴포넌트 다이어그램과 ADR을 만든 프롬프트

/speckit.plan에서 라우트·도메인·Store 계층을 C4로 그리고 그 설계 결정을 ADR로 기록해 달라고 요청합니다

실습

1 어느 단계에서

`/speckit.plan` 계층 구조를 C4 컴포넌트로 그리고 설계 결정을 ADR로 남깁니다

2 이렇게 요청합니다

구조 시각화와 결정 기록을 함께 요청하는 지시

```
> 라우트-도메인-Store 계층을 C4 컴포넌트 다이어그램으로 그리고, 이 plan의 설계 결정을 맥락·결정·대안·결과 네 칸의 ADR로 기록해줘
```

3 이런 산출물이 나옵니다

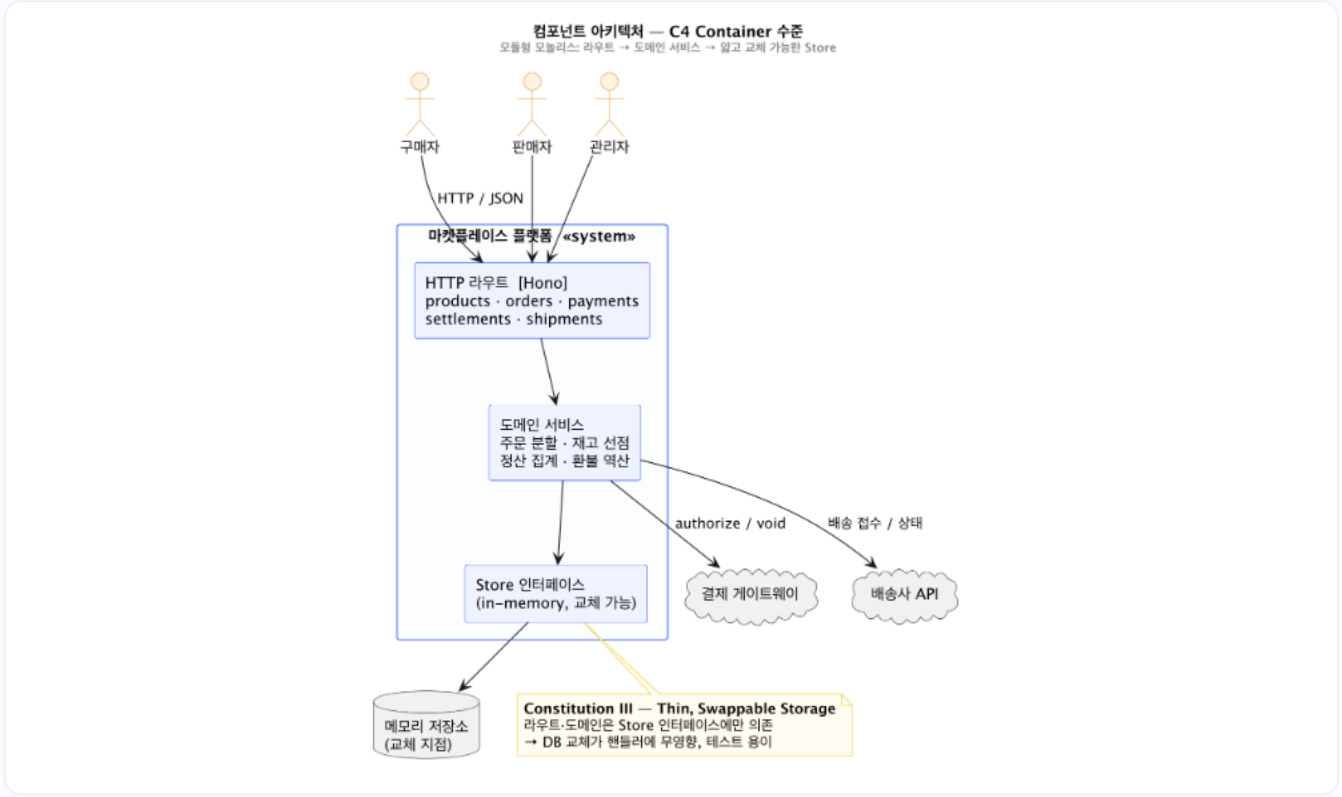
component-architecture.puml
라우트·도메인·Store 계층을 담은 C4 컴포넌트

design/adr/*.md
맥락·결정·대안·결과 네 칸으로 기록한 설계 결정

PLAN ③ · 설명

라우트-도메인-Store 3계층 컴포넌트

설계를 코드 구조로 옮기니 라우트-도메인-Store 3계층과 외부 연동 두 곳으로 구성됐습니다



컴포넌트 상자 = 코드 모듈 · 화살표 = 의존 방향 · 구름 = 외부 시스템 · 원통 = 저장소

다이어그램 읽기

만든 것 설계를 코드 구조로 옮기니 라우트-도메인-Store 3계층과 외부 연동 두 곳으로 구성됐습니다. 앞 장의 상자·화살표·계층이 실제 모듈로 자리 잡은 결과입니다.

라우트 위에서 아래로 계층이 내려갑니다. 최상단 Hono 라우트가 products·orders·payments·settlements·shipments의 HTTP 진입점입니다.

도메인 가운데 도메인 서비스가 주문 분할·재고 선점·정산 집계·환불 역산의 실제 로직을 담습니다.

Store 하단 원통은 Store 인터페이스이며, 현재 in-memory입니다. 라우트·도메인은 이 인터페이스에만 의존합니다.

외부 우측 구름 두 개가 외부 시스템입니다. 결제 게이트웨이(authorize/void)와 배송사 API(접수/상태)를 도메인이 직접 호출합니다.

PLAN ③ · 심화

계층 구조에 담은 아키텍처 결정

계층과 의존 방향이 결정하는 교체·테스트 비용

ADR-005 얇고 교체 가능한 Store

다이어그램에서: 라우트·도메인 → Store 인터페이스에만 의존

결정

라우트와 도메인이 구체 저장소가 아니라 Store 인터페이스에만 의존하게 만듭니다.

근거

저장소 교체가 핸들러에 영향을 주지 않아야 하고 테스트에서 쉽게 대체돼야 합니다.

기각한 대안

~~핸들러가 in-memory 구현에 직접 의존하면 DB 교체 시 라우트를 모두 고쳐야 합니다.~~

결과

현재 in-memory를 PostgreSQL로 바꿔도 라우트·도메인 코드는 그대로입니다.

모듈형 모놀리스 선택

다이어그램에서: 하나의 <<system>> 안의 계층·모듈

결정

마이크로서비스가 아니라 하나의 배포 단위 안에서 모듈로 나누는 구조를 택합니다.

근거

교육·초기 단계에서는 분산 복잡도보다 명확한 모듈 경계가 더 중요합니다.

기각한 대안

~~처음부터 서비스를 분리하면 네트워크·배포·정합성 복잡도가 조기에 커집니다.~~

결과

모듈 경계가 뚜렷해 나중에 필요한 부분만 서비스로 떼어낼 수 있습니다.

외부 연동을 도메인에서

다이어그램에서: 도메인 → 결제 게이트웨이 · 배송사 API

결정

결제·배송 같은 외부 호출을 라우트가 아니라 도메인 서비스에서 다룹니다.

근거

재시도·멱등·실패 처리 같은 규칙이 도메인 로직과 함께 있어야 일관됩니다.

기각한 대안

~~라우트에서 외부를 직접 호출하면 비즈니스 규칙과 통신 코드가 뒤섞입니다.~~

결과

외부 실패 시 롤백(release·void)이 도메인의 트랜잭션 안에서 처리됩니다.

컴포넌트 다이어그램은 코드의 의존 관계를 보여 주는 지도입니다. Store를 인터페이스 뒤에 둔 한 줄이 교체·테스트 비용을 좌우합니다.

speckit-ecommerce-fullscale/design/adr/ADR.md (ADR-005), constitution 원칙 III

PLAN ③ · 개념

결정을 기록하는 네 칸, ADR

아키텍처 결정 기록의 네 부분: 맥락·결정·결과·기각 대안

ADR

정의

하나의 아키텍처 결정을 왜 내렸는지 남기는 짧은 문서입니다. 코드가 아니라 판단의 근거를 보존합니다.

예시: ADR-001 주문과 SubOrder를 분리한다

맥락

Context

결정이 필요했던 상황과 제약입니다. 어떤 문제 때문에 선택이 생겼는지 적습니다.

예시: 한 주문에 여러 판매자의 상품이 섞인다

결정

Decision

실제로 택한 방안입니다. 무엇을 하기로 했는지 한 문장으로 분명히 적습니다.

예시: 배송·정산·환불의 단위를 SubOrder로 둔다

결과

Consequence

이 결정으로 얻는 점과 감수할 비용입니다. 장점과 비용을 함께 적습니다.

예시: 정산이 판매자별로 독립하지만 분할 로직이 필요하다

Michael Nygard, 'Documenting Architecture Decisions', 2011

PLAN ③ · 심화

산출물 곳곳에 흩어진 다섯 결정

하나의 ADR을 여러 다이어그램에 나누어 담습니다

기능	유스케이스	도메인·ERD	시퀀스	상태머신	컴포넌트
ADR-001 Order·SubOrder 분리 이행 단위를 SubOrder로	✓	✓	✓	✓	—
ADR-002 결제 전 재고 선점 reserve 후 결제	—	✓	✓	✓	—
ADR-003 결제 멱등키 중복 결제·취소 방지	—	✓	✓	✓	—
ADR-004 배송 완료 후 정산 완료가 정산 트리거	✓	—	✓	✓	—
ADR-005 얇은 Store 교체 가능한 저장소	—	—	—	—	✓

speckit-ecommerce-fullscale/design/adr/ADR.md, traceability.md

PLAN ③ · 체크포인트

블록 6 정리 (컴포넌트와 ADR)

- ✔ 컴포넌트 다이어그램의 요소(상자·화살표·계층·외부 시스템)를 C4 네 수준 중 오늘 쓰는 세 수준에서 익혔습니다.
- ✔ 라우트-도메인-Store 3계층으로 나누고 저장소를 교체 지점으로 분리했습니다.
- ✔ 얇은 Store(ADR-005)와 모듈형 모놀리스로 교체·테스트 비용을 낮췄습니다.
- ✔ ADR을 맥락·결정·결과·기각 대안의 네 부분으로 정리하고 다섯 ADR이 유스케이스·도메인·시퀀스·상태 다이어그램에 반영됐음을 확인했습니다.

SECTION

07 요구사항·설계·작업 추적

- 여섯 요구사항이 설계·ADR·스펙으로 빠짐없이 이어졌는지 봅니다
- 추적성이 왜 설계 완결의 증거인지 해석합니다
- 한 요구사항을 이슈·브랜치 단위 작업으로 분해합니다

50분 (블록 7) · /tasks

TASKS · 추적성

산출물 완비 후 연결 상태 점검

방금 만든 것

유스케이스·도메인·ERD·시퀀스·상태·컴포넌트·ADR을 모두 갖췄습니다.

이제 할 것

여섯 요구사항이 설계·ADR·스펙으로 끊김 없이 이어졌는지 매트릭스로 검증합니다.

다음으로 넘길 것

검증이 끝나면 요구사항을 작업으로 분해해 실제 구현으로 넘어갑니다.

TASKS · 개념

추적성 매트릭스를 이루는 다섯 요소

요구사항을 설계·ADR·스펙·작업까지 한 행으로 이어 빠짐없이 연결됐는지 한눈에 검증합니다

추적성 매트릭스

한 행 잇기

요구사항 하나를 설계·ADR·스펙·작업까지 한 행으로 잇는 표입니다. 각 행이 끝까지 채워졌는지 확인해 누락을 찾습니다.

예시: R1 멀티셀러 주문 → 도메인·시퀀스 → ADR-001 → 004-multiseller-order

요구사항

행의 출발점

한 행이 시작되는 지점입니다. 시스템이 만족해야 하는 기능에 R1~R6 번호를 붙여 관리합니다.

예시: R1 멀티셀러 주문 생성·분할, R5 판매자 정산

설계 산출물

요구를 그린 것

그 요구를 표현한 도메인·ERD·시퀀스·상태머신 같은 설계입니다. 요구가 실제로 설계로 옮겨졌는지 보여줍니다.

예시: 도메인(Order/SubOrder), 시퀀스, 상태머신

ADR

얽힌 결정

그 요구와 관련된 아키텍처 결정입니다. 한 결정이 여러 요구에 걸리므로 결정을 바꿀 때 영향 범위를 이 열로 추적합니다.

예시: ADR-001 주문 분할, ADR-004 배송·정산

기능 스펙 / 작업

구현의 접점

구현으로 넘어가는 스펙 번호와 이슈·브랜치 단위 작업입니다. 요구가 코드로 이어지는 마지막 접점입니다.

예시: 004-multiseller-order 스펙, feature/004 브랜치의 T001~T007

Dean Leffingwell, 'Managing Software Requirements', 2003 · 요구사항 추적성

TASKS · 실습

추적성 매트릭스를 만든 프롬프트

/speckit.tasks에서 요구사항·설계·ADR·스펙·작업을 한 표로 묶고 /speckit.taskstoissues로 GitHub 이슈까지 만듭니다

실습

1 어느 단계에서

`/speckit.tasks` 설계 산출물을 근거로 작업(tasks.md)을 분해합니다

2 이렇게 요청합니다

작업 분해 뒤 이어서 입력하는 지시

```
> 요구사항·설계·ADR·스펙·작업을 한 표로 잇는 추적성 매트릭스를 만들어줘
```

이어서 `/speckit.taskstoissues` 매트릭스의 작업을 GitHub 이슈로 자동 등록합니다

3 이런 산출물이 나옵니다

`design/traceability.md`
요구사항·설계·ADR·스펙·작업을 잇는 추적표

GitHub Issues
작업 항목이 이슈로 자동 생성되어 구현 단계로 이어집니다

TASKS · 설명

여섯 요구사항을 설계로 이은 추적성 표

배운 매트릭스로 R1~R6을 그려보니 모든 행이 설계·ADR·스펙으로 채워졌습니다

#	요구사항	설계 산출물	ADR	기능 스펙
R1	멀티셀러 주문 생성·분할	도메인(Order/SubOrder), 시퀀스	ADR-001	004-multiseller-order
R2	결제 전 재고 선점	상태머신, 시퀀스	ADR-002	004-multiseller-order
R3	결제 승인·취소(멱등)	시퀀스, 상태머신	ADR-003	005-payment
R4	배송 처리·완료	유스케이스, 상태머신	ADR-004	006-shipping
R5	판매자 정산(수수료 공제)	도메인(Settlement), 시퀀스	ADR-001·004	007-settlement
R6	환불·정산 역산	상태머신	ADR-004	008-refund

TASKS · 심화

추적성이 증명하는 설계 완결성

매트릭스는 표가 아니라 설계가 코드로 빠짐없이 이어졌다는 증거입니다

누락 없음을 증명

다이어그램에서: 모든 요구사항 행에 설계·ADR·스펙이 채워짐

결정

여섯 요구사항 각각에 대응하는 설계 산출물·ADR·스펙을 한 줄로 잇습니다.

근거

빈 칸이 있으면 설계하지 않았거나 구현하지 않은 요구사항이라는 뜻입니다.

기각한 대안

~~요구사항과 코드를 따로 관리하면 무엇이 빠졌는지 눈으로 확인할 수 없습니다.~~

결과

빈 칸이 곧 할 일 목록입니다. 모두 채워지면 설계가 완결됩니다.

변경 영향 추적

다이어그램에서: ADR 열이 여러 요구사항 행에 걸침

결정

하나의 ADR이 어느 요구사항·스펙에 걸리는지 열로 연결해 둡니다.

근거

결정을 바꿀 때 어디까지 영향이 번지는지 미리 알아야 합니다.

기각한 대안

~~결정과 요구사항을 연결하지 않으면 변경 시 영향 범위를 매번 다시 조사합니다.~~

결과

ADR-004를 고치면 R4·R5·R6이 함께 검토 대상임을 즉시 알 수 있습니다.

AI 협업의 근거

다이어그램에서: 요구사항 → 스펙 → 구현 단위로 정렬

결정

요구사항을 스펙·구현 단위로 정렬해 AI에게 무엇을 만들지 명확히 지시합니다.

근거

AI는 흩어진 요구사항보다 정렬된 스펙을 근거로 구현할 때 더 정확합니다.

기각한 대안

~~구두 요구사항으로 바로 지시하면 무엇을 기준으로 검증할지 사라집니다.~~

결과

각 스펙을 테스트와 짝지어 구현이 요구사항을 만족하는지 검증합니다.

추적성 매트릭스는 설계 단계의 마지막 관문입니다. 모든 요구사항이 설계로 이어졌음을 확인해야 구현으로 넘어갑니다.

TASKS · 작업

R1을 이슈·브랜치 단위로 분해

멀티셀러 주문 요구사항이 feature/004 브랜치의 T001~T007 작업이 됩니다

1. **Store 확장 (T001)** — Foundational
   Seller·Inventory·SubOrder 인터페이스와 인메모리 구현을 추가하고 Product에 sellerId를 붙입니다.
2. **US1 테스트 작성 (T002·T003)** — Test-First
   판매자별 SubOrder 분할·서버 총액 계산과 실패 경로(400·404)를 테스트로 먼저 정의합니다.
3. **US1 구현 (T004)** — 구현
   POST /marketplace/orders에서 검증한 뒤 sellerId로 그룹핑하고 SubOrder로 분할해 서버가 subtotal·total을 계산합니다.
4. **US2 선점·취소 (T005·T006)** — reserve, 409
   전량 available 확인 후 원자적 reserve(부족 시 409·무선점), 취소 시 release로 재고를 복원합니다.
5. **Polish (T007)** — 회귀 제로
   전체 vitest 스위트 무회귀를 확인하고 도메인 용어가 다이어그램과 일치하는지 점검합니다.

TASKS · 체크포인트

블록 7 정리 (추적성과 작업 분해)

- ✔ 추적성 매트릭스가 요구사항·설계·ADR·스펙·작업을 한 행으로 잇는 표임을 개념부터 익혔습니다.
- ✔ 여섯 요구사항이 설계·ADR·스펙으로 빠짐없이 이어졌음을 매트릭스로 확인했습니다.
- ✔ 빈 칸이 곧 할 일이며 모두 채워지면 설계가 완결됩니다.
- ✔ 하나의 ADR이 여러 요구사항에 걸려 변경 영향을 추적할 수 있습니다.
- ✔ R1을 T001~T007로 분해해 feature/004 브랜치의 구현 단위를 만들었습니다.

SECTION

08 이슈에서 머지까지

- 이슈에서 브랜치와 PR, 리뷰를 거쳐 머지에 이르는 사이클을 실제로 한 바퀴 돕니다
- 다섯 기능의 브랜치·PR·머지 히스토리를 실제 기록으로 확인합니다
- 설계가 코드보다 먼저라는 하루의 메시지를 정리합니다

50분 (블록 8) · /implement

IMPLEMENT · 구현 사이클

설계·추적성 다음, 실제 GitHub 한 바퀴

방금 만든 것

설계 산출물과 추적성 매트릭스까지 모두 완성했습니다.

이제 할 것

R1을 이슈에서 브랜치와 PR, AI 리뷰를 거쳐 머지까지 GitHub에서 실제로 한 바퀴 돌립니다.

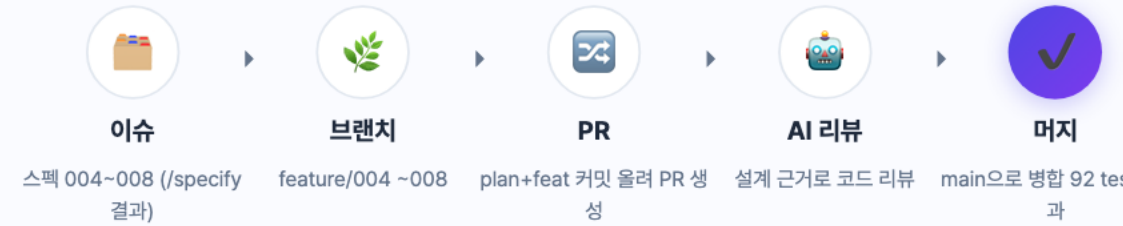
다음으로 넘길 것

이 사이클을 돌고 나면 저장소 자체가 오늘의 결과물이 됩니다.

IMPLEMENT · GITHUB 사이클

한 기능이 저장소를 도는 다섯 단계

스펙 이슈에서 시작해 feature 브랜치·PR·AI 리뷰를 거쳐 main으로 머지됩니다

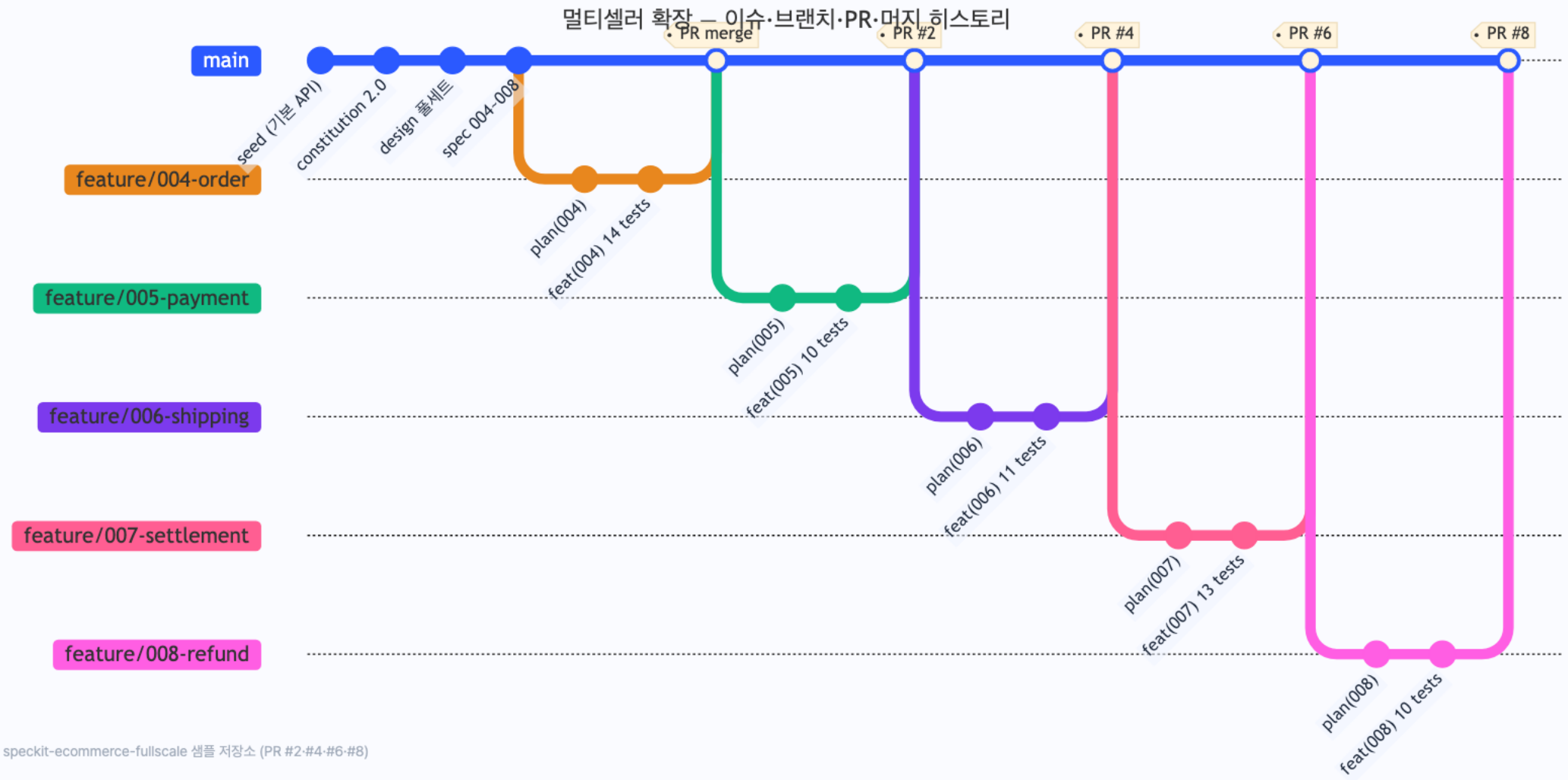


🔗 다섯 기능이 모두 이 사이클을 돌아 PR #2·#4·#6·#8로 머지되었습니다. 라이브에서는 R1을 직접 한 바퀴 돌립니다.

IMPLEMENT · 브랜치·머지 흐름

다섯 기능의 실제 브랜치·머지 히스토리

seed에서 시작해 기능마다 브랜치를 떼고 PR로 main에 병합한 실제 그래프



IMPLEMENT · 설계가 저장소에 남는다

저장소에 그대로 남는 설계 산출물

각 GitHub 아티팩트가 어떤 설계 단계에 대응하는지 보여 줍니다

01 스펙 = 이슈

/specify 결과가 이슈로 올라갑니다. 요구사항과 관찰 가능한 동작이 이슈 본문에 남습니다.

02 작업 = 브랜치

/tasks에서 나눈 작업이 feature/004~008 브랜치가 됩니다. 브랜치가 곧 기능 단위입니다.

03 리뷰 = PR

plan·feat 커밋을 올린 PR에서 설계 근거로 코드를 리뷰합니다. PR #2·#4·#6·#8이 그 기록입니다.

04 추적성 = 머지

머지된 main은 설계와 테스트가 이어진 완결 상태입니다. 92개 테스트가 이를 지킵니다.

📦 저장소 전체가 오늘의 결과물입니다. 다이어그램·ADR·이슈·PR·테스트가 한곳에서 설계의 이력을 증명합니다.

SECTION

09 멀티 도메인·MSA와 코드 조직 심화

- 단일 도메인이 여러 서비스로 갈릴 때 무게중심이 경계 계약과 분산 정합성으로 이동합니다
- 계약과 소비자 주도 테스트, 사가 보상으로 서비스 간 정합성을 설계합니다
- 멀티 레포를 담는 코드 조직과 AI 에이전트 컨텍스트 관리를 다룹니다

심화 · 학습 목표 2개

단일 도메인에서 멀티 도메인으로

서비스가 갈리면 설계의 무게중심이 코드 내부 구조에서 경계의 계약과 분산 정합성으로 옮겨갑니다

단일 도메인

오늘 만든 단일 레포 API

- 하나의 코드베이스와 하나의 데이터베이스
- 함수 호출로 주문·결제·재고가 이어짐
- 무게중심은 코드 내부 구조와 모듈 경계
- 한 트랜잭션으로 일관성 보장(ACID)

전환

멀티 도메인·MSA

서비스로 갈린 구조

- 주문·결제·배송·정산이 독립 서비스로 분리
- 서비스 간 계약(API·이벤트)이 통신을 지배
- 무게중심은 경계 계약과 분산 정합성
- 서비스별 DB라 사가와 보상이 필요

계약이 되는 서비스 경계

동기 호출은 OpenAPI, 비동기 이벤트는 AsyncAPI로 정의하고 버전을 명시해 관리합니다

동기 계약

OpenAPI

서비스 간 동기 호출(요청과 응답)의 단일 기준입니다. 오늘 그린 단일 API 명세가 서비스가 갈리는 순간 팀 간 계약으로 승격됩니다.

예시: 주문 서비스가 결제 서비스의 /payments 엔드포인트를 호출하는 규약

비동기 계약

AsyncAPI

이벤트와 메시지 기반 비동기 경계를 별도로 명세합니다. 발행하는 이벤트의 형식과 의미를 계약으로 고정합니다.

예시: 결제 완료 이벤트(PaymentApproved)를 재고·정산 서비스가 구독

호환성

버전 관리

계약은 v1, v2처럼 버전을 명시해 관리하고 기존 버전을 깨뜨리는 변경을 금지합니다. 백워드 호환성 검사를 CI에서 미리 수행합니다.

예시: 필드 삭제 대신 v2를 신설하고 구버전 소비자는 v1을 유지

소비자 주도 계약 테스트

소비자가 먼저 기대를 정의하고, 제공자는 배포 전 can-i-deploy로 계약 충족을 확인합니다



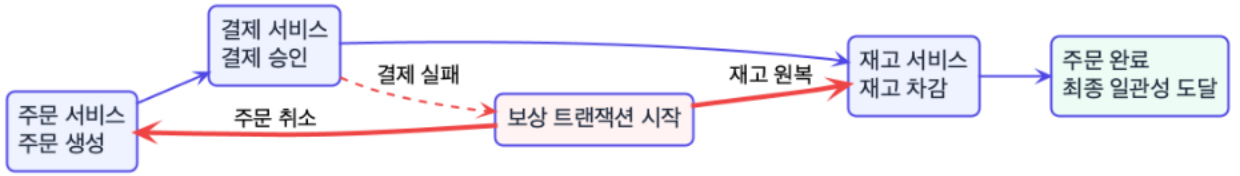
🔗 값비싸고 취약한 E2E 통합 테스트를 대체해 팀별 독립 배포를 실현합니다. 다만 계약은 인터페이스 구조만 검증하며 행위와 성능까지 보장하지는 않습니다.

Pact (Consumer-Driven Contract), Specmatic, Martin Fowler

멀티 도메인·MSA 심화

분산 트랜잭션은 사가와 보상으로

서비스별 DB가 분리되면 2PC가 실용적이지 않아, 로컬 트랜잭션 연쇄와 보상으로 최종 일관성을 택합니다



실선은 정상 사가 흐름, 굵은 붉은 선은 보상(되돌리기) 흐름입니다.

다이어그램 읽기

정상 흐름
주문 생성, 결제 승인, 재고 차감이 각 서비스의 로컬 트랜잭션으로 이어집니다.

결제 실패
실패해도 자동 롤백이 없어 보상 트랜잭션을 명시적으로 시작합니다.

보상 체인
재고 원복과 주문 취소로 앞 단계를 도메인 규칙에 맞게 되돌립니다.

조율 방식
중앙 조정자가 지휘하는 오케스트레이션과 이벤트가 전파되는 코레오그래피로 나뉩니다.

microservices.io Saga, Microsoft Azure 'Compensating Transaction'

멀티 도메인 명세의 계층화

도메인 마스터 명세와 서비스별 명세로 나누고 REQ·SVC 같은 ID로 잇습니다. 명세의 구속력은 서비스마다 다르게 정합니다

구속력 낮음

spec-first

명세로 시작하되 이후 폐기를 허용합니다. 빠르게 방향을 실험하는 서비스에 적합합니다.

예시: 실험적 추천 서비스

구속력 중간

spec-anchored

코드와 명세를 병행하고 테스트로 정렬을 유지합니다. 대부분의 서비스에 기본으로 둡니다.

예시: 핵심 결제·정산 서비스

구속력 높음

spec-as-source

사람은 명세만 다루고 코드는 전량 생성합니다. 명세가 유일한 기준이 됩니다.

예시: 정형화된 CRUD 서비스

코드 조직·AI 컨텍스트 설명

코드 조직 세 방식

멀티 레포와 모노레포, 그 사이의 서브모듈을 두고 AI 에이전트가 선택 기준을 바꿉니다

방식	특징	트레이드오프
멀티레포	서비스마다 독립 레포	독립 배포와 소유가 명확하나, 크로스 변경과 버전 조율이 어렵습니다
모노레포	한 레포에 여러 패키지	원자적 크로스 변경과 공유가 쉽지만, 전용 도구(Nx·Turborepo)가 필요합니다
서브모듈	메인 레포에 하부 서브모듈	멀티 레포를 하나처럼 다루는 대표 방식으로, 독립 이력과 단일 작업 공간을 함께 가집니다

AI 에이전트에게 레포 경계는 컨텍스트 벽으로 작동합니다. 단일 인덱싱 작업 공간이 유리하다는 2026 실무 논의가 있으며, 크로스 레포를 자동 조율하는 도구는 아직 미성숙합니다.

dortort 'Monorepo vs Multi-Repo: Why AI Agents Tip the Scale' (실무 블로그)

코드 조직·AI 컨텍스트 설명

Claude Code의 멀티 레포 컨텍스트 3축

실행 위치와 CLAUDE.md 계층, 디렉터리 확장으로 에이전트가 읽는 범위를 통제합니다

1 **실행 위치**
어디서 실행하느냐가 파일 접근과 로드되는 CLAUDE.md를 결정합니다. 레포 루트는 전체 접근, 하위 디렉터리는 그 서브트리와 모든 조상 CLAUDE.md입니다.
작업 범위

2 **CLAUDE.md 계층화**
루트에 전역 규칙을 두고, 패키지나 서브시스템(src/api, src/db)마다 규약을 두어 지금 작업하는 코드의 규칙만 로드합니다.
규칙 로드

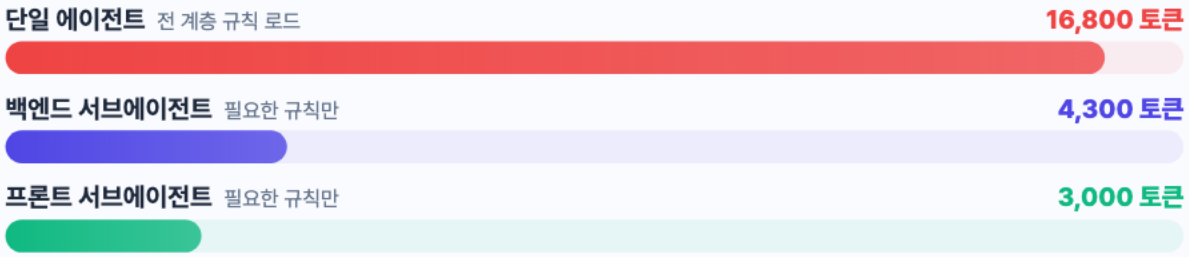
3 **디렉터리 확장**
additionalDirectories 설정이나 --add-dir 플래그로 형제 패키지와 별도 레포까지 한 세션에서 읽고 편집합니다. 필요할 때만 범위를 넓힙니다.
--add-dir

Claude Code 공식 문서 'large codebases'

코드 조직·AI 컨텍스트 심화

서브에이전트로 컨텍스트 분할

격리된 세션이 필요한 규칙만 새로 읽어, 코드를 읽기 전 생기는 규칙 오버헤드를 줄입니다



한 사례에서

17개 프로젝트 모노레포에서 규칙 오버헤드가 약 4배 줄었다는 보고입니다. 오케스트레이터가 서브에이전트를 순차·병렬로 조합합니다. 오늘 이 강의안도 그렇게 만들었습니다.

dortort 실무 블로그, addyosmani 'code agent orchestra'

코드 조직·AI 컨텍스트 설명

멀티 레포에서 명세와 추적성

명세는 코드와 같은 레포에 두고, ID 규약과 CI 게이트로 크로스 레포를 이어 붙입니다

ℹ️ INFO **명세 co-location**

명세는 대개 코드와 같은 레포에 둡니다. 크로스 레포 추적성 도구가 아직 미성숙하기 때문입니다.

REQ·SVC 같은 ID 규약과 CI 게이트로 서비스 사이를 잇습니다.

⚠️ WARNING **컨텍스트 저하**

긴 멀티 레포 세션에서는 에이전트가 현재 레포를 잊고 보안 요구를 누락하는 문제가 보고됩니다.

실행 위치와 CLAUDE.md 계층으로 컨텍스트를 좁히는 것이 필수입니다.

💡 TIP **범위 좁히기**

관련 서비스 명세만 주고 필요할 때만 디렉터리를 넓힙니다. 좁은 컨텍스트가 출력 품질을 지킵니다.

서브에이전트로 도메인별 세션을 분리하면 규칙 로드가 가벼워집니다.

MSA·멀티 레포 심화 · 체크포인트

블록 9 정리 (경계 계약과 코드 조직)

- ✔ 서비스가 갈리면 OpenAPI·AsyncAPI 계약이 1급 산출물이 되고, 소비자 주도 계약 테스트가 팀별 독립 배포를 지원합니다.
- ✔ 분산 트랜잭션은 사가와 보상으로 설계하며, 보상은 단순 복원이 아니라 도메인 규칙에 맞는 되돌리기입니다.
- ✔ 명세는 도메인 마스터와 서비스별로 계층화하고 구속력(spec rigor)을 서비스마다 정합니다.
- ✔ 코드 조직 세 방식의 트레이드오프를 이해하고 Claude Code의 컨텍스트 3축과 서브에이전트로 멀티 레포 작업을 통제합니다.

CLOSING · 회고

오늘 하루의 한 줄

AI 시대에도 설계가 코드보다 먼저입니다. 코딩은 설계의 결과일 뿐입니다.

Spec Kit 실전 설계

멀티셀러 마켓플레이스 설계 산출물 전체

> 유스케이스에서 정산·환불까지, 각 산출물을 읽고 왜 그렇게 설계했는지 해석했습니다. 다이어그램이 담은 결정이 곧 구현의 근거가 됩니다.

CLOSING · 마무리

정리 · 설계가 코드보다 먼저

- ✔ Spec Kit 5단계마다 설계 산출물을 만들고 다이어그램으로 시각화했습니다.
- ✔ 산출물마다 무엇이 그려졌고 왜 그렇게 그렸는지 읽고 해석했습니다.
- ✔ 다섯 ADR과 추적성 매트릭스로 설계 결정과 요구사항을 빠짐없이 이었습니다.
- ✔ 이슈에서 브랜치, PR, 머지로 이어지는 사이클로 설계를 저장소에 그대로 남겼습니다(92 tests).
- ✔ 멀티 도메인·MSA와 코드 조직 심화로 서비스를 나눌 때의 설계까지 넓혔습니다.